Will We Eat the Rich If We Run Out of Cake?: Analysing the Cost of Living's Impact on Crime Rates for the City of Chicago

1. Background, Aims, and Objectives

1.1 Introduction

Historically, research studies have linked economic inequality to crime rates, as those who are economically disadvantaged find incentive in pursuing illegal activities (Becker 1968; Kang 2016).

It is reasonable to assume that rising costs of living would contribute to increased economic inequality and thus an increase in crime rates. In 2022, inflation experienced its highest rate of increase in 40 years, surging by 7.5% (Fuscaldo, 2022). Despite this exponential increase in costs, contrary to what one might think, 'new statistics suggest the cost of living crisis has not had a major impact on crime overall, despite greater financial pressures' (Nicholson, 2022).

Could it be that higher living costs don't necessarily influence total crime rates, but rather lead to a rise in economically driven crimes, while other types of crime decrease? Or, in this modern age where 'eat the rich' is a commonly utilized phrase, are we perhaps all talk and no criminal action?

1.2 Research Aims and Objectives

1.2.2 Research Aim

In what ways, if any, does the cost of living influence crime rates in Chicago between the years 2010 and 2022?

1.2.2 Objectives

1. To Analyze the Trends in Cost of Living and Crime Rates Over Time

1.1 Examine changes in various cost of living indicators (such as housing, basic amenities, food, transportation, and entertainment costs) from 2010 to 2022.

1.2 Analyze the trends in different types of crime rates in the same period.

1.3 Determine if there are any notable correlations between the trends in cost of living and crime rates.

2. To Identify Specific Cost of Living Factors Most Strongly Associated with Crime Rates

2.1 Conduct a detailed statistical analysis to identify which elements, if any, of the cost of living (housing costs, basic necessities, entertainment, etc.) are most strongly correlated with various types of crimes.

Basic Project Objectives:

1. Select appropraite resources needed to address the research question and objectives
2. Utilise data collection and cleaning techniques to ensure that the data is ready for analysis
3. Conduct basic exploratory analysis to identify trends and insights that may be helpful when conducting a more detailed advanced analysis

1.3 Data

1.3.1 Data Requirements

The data needed for this research project falls into two primary categories: crime data and cost of living data.

1. Crime Data: This dataset should encompass comprehensive crime statistics for the city of Chicago over the specified study period (in this case, 2010-2022). Essential elements needed for analysis include the type of crime committed and the year the crime was committed. It could be helpful if the type of crime was categorized based on its relevance to economic factors.

2. Cost of Living Data: To analyze the impact of economic conditions on crime rates, a detailed dataset on the cost of living in Chicago during the same period (2010- 2022) is required. This should include basic cost of living elements such as housing costs (both rental and purchase prices), cost of basic necessities (like food and utilities), and prices of other common goods and services/ activities. These also need to be seperated by year as I plan on using the year to combine the data sets if needed.

1.3.2 Data Sets and Web Scraping

1. Sourced Data Set: Chicago Crimes: Crimes - 2001 to Present
   The Chicago Crimes Data Set can be found here, https://catalog.data.gov/dataset/crimes-2001-to-present. This Data set has statistics for the years 2001 until 2022.

   This data set is 1,825,797 KB and contains the following information:
   'ID', 'Case Number', 'Date', 'Block', 'IUCR', 'Primary Type','Description', 'Location Description', 'Arrest', 'Domestic', 'Beat','District', 'Ward', 'Community Area', 'FBI Code', 'X Coordinate','Y Coordinate', 'Year', 'Updated On', 'Latitude', 'Longitude','Location'.

2. Web Scraped Data: Historical Prices in Chicago, IL 2010-2022
   The Chicago Cost of Living Data Set was obtained through web scraping from Numbeo, a comprehensive website containing inoformation on living expenses. The data, which covers the years 2010 to 2022, can be accessed at Numbeo's Chicago Cost of Living Page, https://www.numbeo.com/cost-of-living/city-history/in/Chicago. This site offers a detailed look into various aspects of living costs in Chicago, encapsulating a variety of economic indicators.

   The site includes information on 'Year', 'Average Monthly Net Salary (After Tax)', 'Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate', 'Apartment (1 bedroom) in City Centre', 'Apartment (1 bedroom) Outside of Centre', 'Apartment (3 bedrooms) in City Centre', 'Apartment (3 bedrooms) Outside of Centre', 'Price per Square Meter to Buy Apartment in City Centre', 'Price per Square Meter to Buy Apartment Outside of Centre', 'Basic (Electricity, Heating, Cooling, Water, Garbage) for 85m2 Apartment', 'Internet (60 Mbps or More, Unlimited Data, Cable/ADSL)', 'Mobile Phone Monthly Plan with Calls and 10GB+ Data', 'One-way Ticket (Local Transport)', 'Gasoline (1 liter)', 'Monthly Pass (Regular Price)', 'Meal, Inexpensive Restaurant', 'Meal for 2 People, Mid-range Restaurant, Three-course', 'McMeal at McDonalds (or Equivalent Combo Meal)', 'Domestic Beer (0.5 liter draught)', 'Imported Beer (0.33 liter bottle)', 'Coke/Pepsi (0.33 liter bottle)', 'Water (0.33 liter bottle)', 'Cappuccino (regular)', 'Apples (1kg)', 'Oranges (1kg)', 'Potato (1kg)', 'Lettuce (1 head)', 'Rice (white), (1kg)', 'Tomato (1kg)', 'Banana (1kg)', 'Onion (1kg)', 'Local Cheese (1kg)', 'Bottle of Wine (Mid-Range)', 'Cigarettes 20 Pack (Marlboro)', 'Chicken Fillets (1kg)', 'Beef Round (1kg) (or Equivalent Back Leg Red Meat)', 'Milk (regular), (1 liter)', 'Loaf of Fresh White Bread (500g)', 'Eggs (regular) (12)', '1 Pair of Jeans (Levis 501 Or Similar)', '1 Summer Dress in a Chain Store (Zara, H&M, ...)', '1 Pair of Nike Running Shoes (Mid-Range)', '1 Pair of Men Leather Business Shoes', 'Fitness Club, Monthly Fee for 1 Adult', 'Tennis Court Rent (1 Hour on Weekend)', and 'Cinema, International Release, 1 Seat'.

   This detailed and multifaceted website provides an extensive view into the economic landscape of Chicago from 2010-2022, which is essential for comprehensive analysis.

   3. Web scraped inflation rate data: After web scraping Numbeo data, I realized that I needed to fill in missing variables. This posed a problem, as filling in with the mean or median would skew the data as the price increases over the years, so backfilling these values was challenging.
   I decided the best course of action was to use the inflation rate from 2010-2022 to calculate the missing values using the known prices. This would allow for the most accurate data. So, I needed to source this inflation rate data. I did so at the site: https://www.usinflationcalculator.com/inflation/current-inflation-rates/

   Web scraping this data falls under fair use principles. The data scraped is factual, consisting of historical inflation figures, which are not copyrighted. The site also does not disallow web scraping as shown through the lack of mention of web scraping in the terms of service.
   I've only extracted raw data, not the website's formatting or design elements. This distinction is crucial in ensuring no copyright infringement. The use of this data is purely for academic purposes, not commercial, further proving this falls under

fair use. Web scraping this publicly
available information was crucial for the educational value of my project.

1.3.3 Data Limitations and Constraints

1. Data Synchronization: The time frames of the Chicago Crime Data
and the Cost of Living Data may not align perfectly. In this case,
the Chicago Crime Data is from the years 2001 until 2022 and the
Cost of Living Data is from 2010- 2022. As the data sets cover different
periods of time, it could pose challenges in correlating changes in crime
rates with shifts in cost of living and will need to be addressed during
the data cleaning process.

2. Reliability of Crowd-Sourced Data: Since the Cost of Living Data
gathered on the Numbeo website is crowd-sourced, it may not always be
entirely accurate or representative of the entire population of Chicago.
The subjective nature of some of these data points might introduce biases.

3. Crime Data Limitations: Crime data may have gaps or inconsistencies,
especially in underreported categories of crime. Additionally, changes in
law enforcement practices over time can affect the number and types of
crimes recorded. For example, on January 1, 2020, Chicago legalized the
recreational use of marijuana. This, alone, could skew data in regard to
drug related crime.

4. Variability in Cost of Living Components: The Cost of Living Data encompasses
a wide range of variables, from housing costs to the price of consumer goods.
The variability and fluctuation in these elements can make it challenging to derive
their direct impact on crime rates.

5. Data Aggregation Levels: The aggregation level of data in both data sets
could limit analysis. For example, if crime data is not categorized by
economic-related crimes (like theft and burglary) specifically,
it may be hard to draw direct correlations with economic factors.
However, this can be addressed during the data cleaning process.

6. External Influencing Factors: Both sets of data could be influenced by
external factors like policy changes, economic shifts, social movements,
or major events (like Covid-19), which are not directly accounted for within
the data sets but could significantly impact both the cost of living and crime rates.
As mentioned earlier, policiy changes, such as legalizing marijuana in 2020
and Covid-19 interrupting activities in 2020-2022, could skew the data
drastically and will need to be accounted for in the final anaylsis.

7. Resource and Technical Constraints: The size and complexity of the data sets
might require substantial computational resources for analysis.
Additionally, the technical expertise required to effectively clean, process,
and analyze the data could pose a constraint as this researcher is quite new to
Python, SQL, and R.

1.4 Ethics

There are several ethical considerations needed for this research project. Any crime data
acquired needs to be anonymous. The crime data sourced from Data.gov is anonymous and does
not have any information that could be used to identify victims. The accuracy of the data
especially the crowd-sourced cost of living information, must be critically assessed to
avoid misinterpretation or overgeneralization. Compliance with legal and ethical standards for
data use and reporting is also necessary.

1.4.1 Chicago Crime Data Set

The Chicago Crimes Data Set accessed through Data.gov is a public dataset and is permissible
for this use in research.

1.4.2 Chicago Cost of Living Numbeo

The data gathered through web-scraping Numbeo is permissable according to Numbeo's
terms of use. Numbeo's terms of use include personal use and academic research. For personal
and academic purposes, Numbeo requires appropriate credit and a link back to their website,
a guideline I shall rigorously follow to respect intellectual property rights.

1.4.3 Future Use

Use of the original datasets will need to adhere to the terms and conditions set by
the original source. Datasets derived from these original sources will be held to
the same terms of use as the originals. All analysis and conclusions are my own,
but may be used for further research endevours by other academics.

1.5 Imported Libraries

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
import requests
from bs4 import BeautifulSoup
from functools import reduce
import numpy as np
```

1.6 Chicago Crimes Data Set

Accessed via https://catalog.data.gov/dataset/crimes-2001-to-present To begin, I will examine the contents of the data set to guide future data cleaning.

In [2]:
```
crime_data_large = pd.read_csv('Crimes_-_2001_to_Present.csv')

crime_data_large.head(3)
```

Out[2]:

	ID	Case Number	Date	Block	IUCR	Primary Type	Description	Location Description	Arres
0	11037294	JA371270	03/18/2015 12:00:00 PM	0000X W WACKER DR	1153	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT OVER $ 300	BANK	Fals
1	11646293	JC213749	12/20/2018 03:00:00 PM	023XX N LOCKWOOD AVE	1154	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT $300 AND UNDER	APARTMENT	Fals
2	11645836	JC212333	05/01/2016 12:25:00 AM	055XX S ROCKWELL ST	1153	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT OVER $ 300	NaN	Fals

3 rows × 22 columns

In [3]:
```
#I want to be sure the date column is in a useable format
crime_data_large['Date'] = pd.to_datetime(crime_data_large['Date'], format='%m/%d/%
```

In [4]:
```
crime_data_large.head(3)
```

Out[4]:

	ID	Case Number	Date	Block	IUCR	Primary Type	Description	Location Description	Arrest
0	11037294	JA371270	2015-03-18 12:00:00	0000X W WACKER DR	1153	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT OVER $ 300	BANK	False
1	11646293	JC213749	2018-12-20 15:00:00	023XX N LOCKWOOD AVE	1154	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT $300 AND UNDER	APARTMENT	False
2	11645836	JC212333	2016-05-01 00:25:00	055XX S ROCKWELL ST	1153	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT OVER $ 300	NaN	False

3 rows × 22 columns

1.6.1 Dimensionality Reduction

As this data set is quite large and contains data for the years 2001 to 2022, I will be reducing the data to the years 2010 to 2022 to examine the recent historic trends. This also conforms to the years of the webscraped data that is brought in later on.

In [5]:
```
#Crime_data_1022 is a new data frame that now only contains crime data for the year
crime_data_1022 = crime_data_large[crime_data_large['Date'].dt.year.between(2010, 2
```

1.6.1.1 Examining the Data in the crime_data_1022 dataset

Now that I've trimmed down the data to only include data for the years 2010-2022,
I will take a look at the various columns and, more specifically, the types
of crimes that are listed in the data set to better understand what analysis
I may be able to conduct using this data.

In [6]:
```
unique_primary_types = sorted(crime_data_1022['Primary Type'].unique())
print(unique_primary_types)
```

```
['ARSON', 'ASSAULT', 'BATTERY', 'BURGLARY', 'CONCEALED CARRY LICENSE VIOLATION',
'CRIM SEXUAL ASSAULT', 'CRIMINAL DAMAGE', 'CRIMINAL SEXUAL ASSAULT', 'CRIMINAL TRE
SPASS', 'DECEPTIVE PRACTICE', 'GAMBLING', 'HOMICIDE', 'HUMAN TRAFFICKING', 'INTERF
ERENCE WITH PUBLIC OFFICER', 'INTIMIDATION', 'KIDNAPPING', 'LIQUOR LAW VIOLATION',
'MOTOR VEHICLE THEFT', 'NARCOTICS', 'NON - CRIMINAL', 'NON-CRIMINAL', 'NON-CRIMINA
L (SUBJECT SPECIFIED)', 'OBSCENITY', 'OFFENSE INVOLVING CHILDREN', 'OTHER NARCOTIC
VIOLATION', 'OTHER OFFENSE', 'PROSTITUTION', 'PUBLIC INDECENCY', 'PUBLIC PEACE VIO
LATION', 'RITUALISM', 'ROBBERY', 'SEX OFFENSE', 'STALKING', 'THEFT', 'WEAPONS VIOL
ATION']
```

In [7]:
```
print(crime_data_1022.columns)
```

```
Index(['ID', 'Case Number', 'Date', 'Block', 'IUCR', 'Primary Type',
       'Description', 'Location Description', 'Arrest', 'Domestic', 'Beat',
       'District', 'Ward', 'Community Area', 'FBI Code', 'X Coordinate',
       'Y Coordinate', 'Year', 'Updated On', 'Latitude', 'Longitude',
       'Location'],
      dtype='object')
```

1.6.2 Feature Selection

After examining the different columns in the data set, I can see that there are several of which are not applicable to my research questions or offer extraneous data. As I am interested in types of crimes and number of crimes committed in Chicago between the years of 2010 and 2022, I need to focus on the data that will enhance this study. So, columns such as 'IUCR', 'Beat', 'Updated On', 'Latitude', 'Longitude', 'Location', 'X Coordinate', and 'Y Coordinate'are not needed for this study and will therefore be removed to create a smaller data set. This way, my dataset will include the relevant information for this study which is the year, the type of crime committed and the district it was committed in.

- 'IUCR':The IUCR is the "Illinois Uniform Crime Reporting" code which denotes what type of crime occurred in a numerical code. However, I have the column, 'Primary Type' which tells me the type of crime that occurred so this column is redundant for my purposes.
- 'Beat': This column gives which number of police beat the crime took place in. This is a small geographical area and is too narrow for my purposes. I still have the columns District, Ward, and Community Area which give me locations of the crimes committed.
- 'X Coordinate' and 'Y Coordinate': These are meant to give exact locations and are too narrow for my purposes nor is the data available.
- 'Updated On':It is not neccessary for me to know when the entry was updated, rather to know that a crime did in fact take place.
- 'Latitude' and 'Longitude':These are also too narrow spatially and is not needed for this study as I am able to use District etc. to categorize by location if needed.
- 'Ward': As I have the district, the ward is not neccessary
- 'Community Area': As I have the district, the community area is not needed.
- 'Location':As I have already dropped Latitude and Longitude, this column is also unneccesary.
- 'Location Description': This describes the location of the crime as in in the sewer or in a school. This information is unneccessary as the exact location is not pertinenet when examining the cost of living's effect on crime rate.
- 'FBI Code': Unneccessary for my analysis as I will not be looking at FBI statistics

In [8]:
```
columns_to_drop = ['IUCR', 'Beat', 'Updated On', 'Latitude', 'Longitude', 'Locatior

crimes_1022_small = crime_data_1022.drop(columns=columns_to_drop)
```

In [9]:
```
print(crimes_1022_small.columns)
```

```
Index(['ID', 'Case Number', 'Date', 'Block', 'Primary Type', 'Description',
       'Arrest', 'Domestic', 'District', 'Year'],
      dtype='object')
```

In [10]:
```
missing_values = crimes_1022_small.isna().sum()
print(missing_values)
```

```
ID                0
Case Number       0
Date              0
Block             0
Primary Type      0
Description       0
Arrest            0
Domestic          0
District          1
Year              0
dtype: int64
```

With only one missing data point in the column 'district', this seems like a reasonably accurate
representation of the data.The geographic element is not as pertinent in this study, as all of
the data is for the city of Chicago which is the main focus.District is not pertinent information
but has been kept in case it is later determined that a micro-level should be examined.

In [11]:
```
print(crimes_1022_small['District'].unique())
```

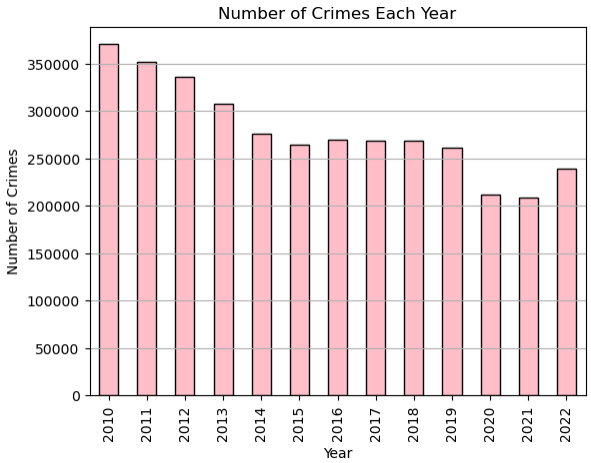
```
[ 1. 25.  8. 17. 22.  6. 15. 19.  2.  9.  4.  7. 12. 16. 18. 10. 11. 14.
 24.  3.  5. 20. 31. nan]
```

1.6.3 Visualisations For Chicago Crime Data

1. Number of Crimes Committed Each Year Graph:

This graph shows that there has been a steady decline in number of crimes committed
from 2010-2022. Noteably, the pandemic years, 2020 and 2021, declined quite a bit more
than in other years. This could require a analysis in which the pandemic years are
negated to avoid bias. However, even without the pandemic years, a decline in crime
rate is still visible.

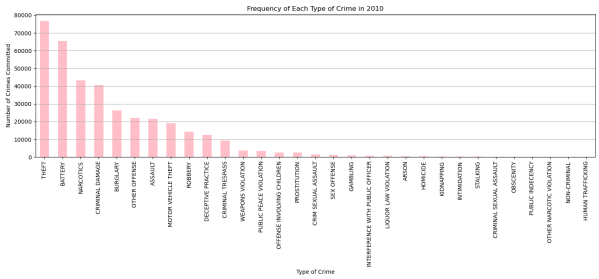
In [12]:
```
#Here, I will take a look at the number of crime committed each year from 2010-2022

yearly_crimes = crimes_1022_small.groupby('Year').size()

yearly_crimes.plot(kind='bar', color='pink', edgecolor='black')
plt.title('Number of Crimes Each Year')
plt.xlabel('Year')
plt.ylabel('Number of Crimes')
plt.grid(axis='y')

plt.show()
```

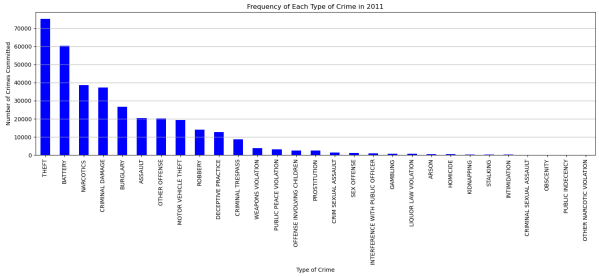


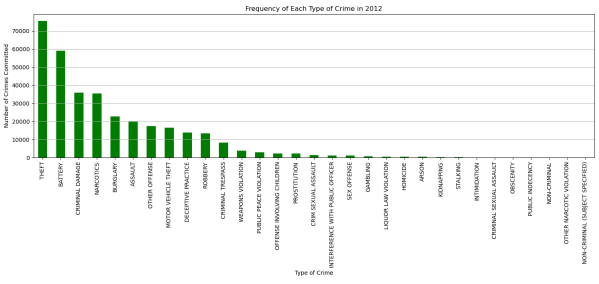
2. Number of Each Type of Crime Committed over the Years 2010- 2022

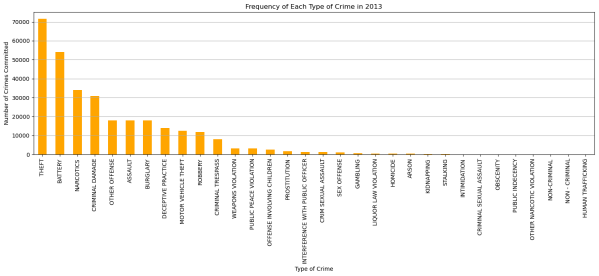
This graph depicts the number of each type of crime committed over the years 2010-2022.
As you can see, there are quite a lot of options for type of crime committed.
Some of which, such as theft, are rather popular choices year after year and others,
such as Arson, are less common. It could be important to categorize these types of crimes
based on how they relate to cost of living or economic hardships. For example,
theft may be related to an increase in cost of living as if someone is no longer
able to afford a neccisity, such as baby formula, he or she may be driven to theivery.

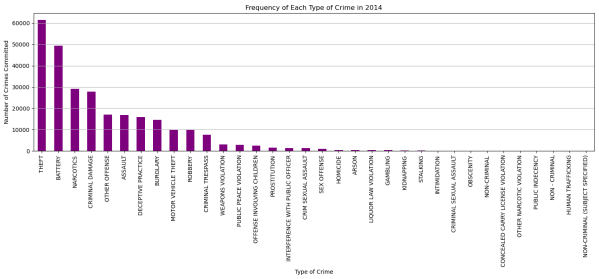
In [13]:
```
colors = ['pink', 'blue', 'green', 'orange', 'purple', 'cyan', 'yellow', 'red', 'br

for year, color in zip(range(2010, 2023), colors):
    crimes_year = crimes_1022_small[crimes_1022_small['Year'] == year]
    crime_counts_year = crimes_year['Primary Type'].value_counts()
    crime_counts_year.plot(kind='bar', figsize=(15,7), color=color)

    plt.title(f'Frequency of Each Type of Crime in {year}')
    plt.xlabel('Type of Crime')
    plt.ylabel('Number of Crimes Committed')
    plt.grid(axis='y')
    plt.tight_layout()
    plt.show()
```

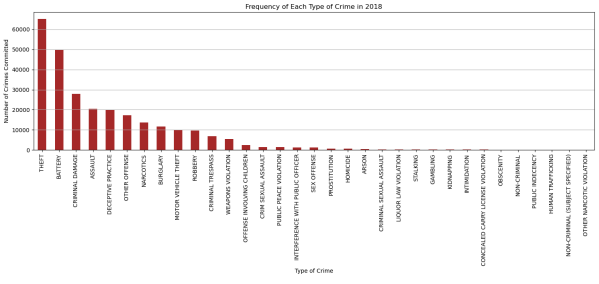


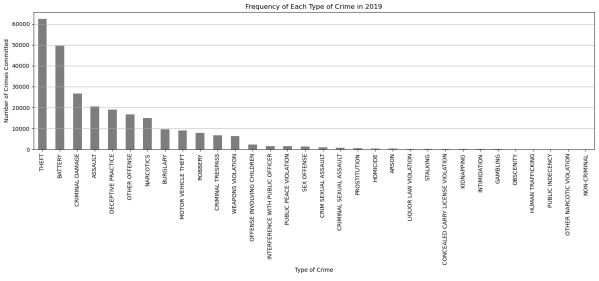


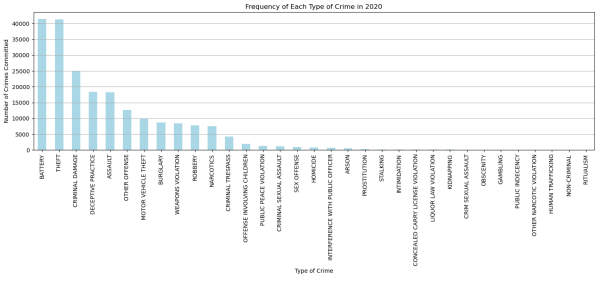


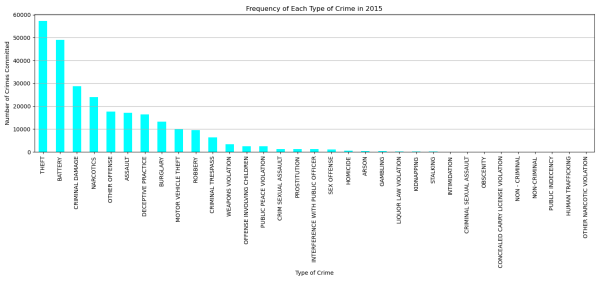


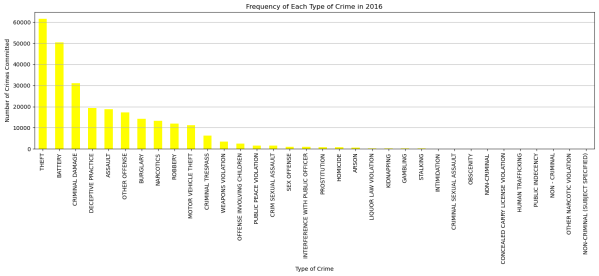


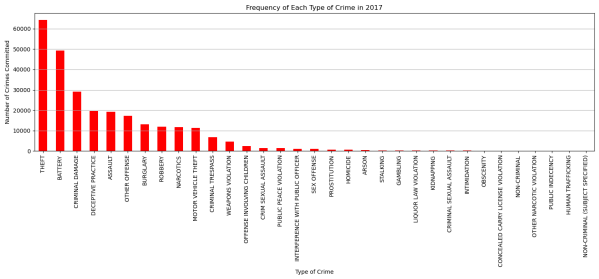


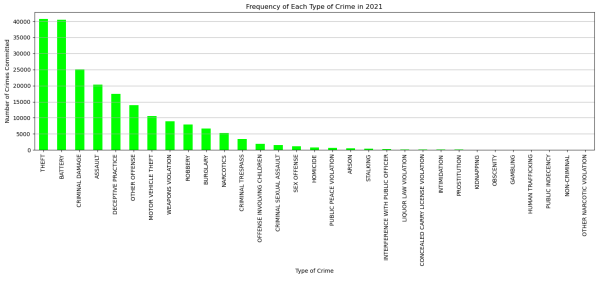


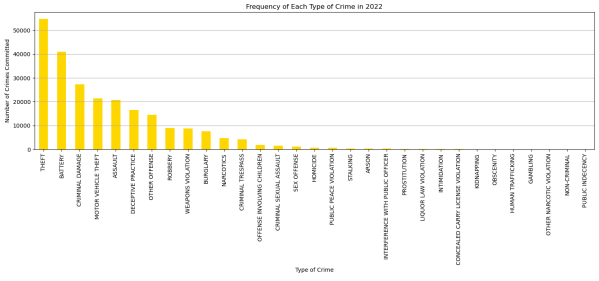












If I am going to categorize types of crimes, I first need to take a look at all
of the unique types of crimes in the data and then group them into categories
based on economic factors.

In [14]:
```
crime_types = crimes_1022_small['Primary Type'].unique()
print(crime_types)
```

```
['DECEPTIVE PRACTICE' 'OTHER OFFENSE' 'THEFT' 'BATTERY' 'ASSAULT'
 'WEAPONS VIOLATION' 'BURGLARY' 'NARCOTICS' 'LIQUOR LAW VIOLATION'
 'CRIM SEXUAL ASSAULT' 'MOTOR VEHICLE THEFT' 'CRIMINAL DAMAGE'
 'OFFENSE INVOLVING CHILDREN' 'CRIMINAL TRESPASS' 'ROBBERY'
 'INTERFERENCE WITH PUBLIC OFFICER' 'SEX OFFENSE' 'PUBLIC PEACE VIOLATION'
 'CRIMINAL SEXUAL ASSAULT' 'PROSTITUTION' 'HOMICIDE' 'STALKING' 'ARSON'
 'CONCEALED CARRY LICENSE VIOLATION' 'GAMBLING' 'OBSCENITY' 'KIDNAPPING'
 'INTIMIDATION' 'OTHER NARCOTIC VIOLATION' 'NON-CRIMINAL'
 'PUBLIC INDECENCY' 'HUMAN TRAFFICKING' 'RITUALISM'
 'NON-CRIMINAL (SUBJECT SPECIFIED)' 'NON - CRIMINAL']
```

3. Now, to categorize the types of crime I will sort each crime type into the following categories using the information in the 'Description' column:

- economically motivated Crimes:
  - Theft, Burglary, Motor Vehicle Theft, Robbery: These are crimes typically committed with the intention of gaining property or money illegally.

- Deceptive Practice: Involves fraud or deception for financial gain.
- Gambling, Prostitution: These are considered economically motivated as they are often undertaken for financial profit.
- Arson: Arson can be ecnonmically motivated if it used for insurance fraud or another financial gain.

- Violent Crimes:
  - Battery, Assault, Criminal Sexual Assault, Homicide, Kidnapping, Intimidation, Sex Offense, Stalking: These are direct crimes against individuals, involving physical harm or the threat of harm.
- Drug Related Crime:
  - Narcotics, Other Narcotic Violation: These are directly related to the illegal trade, manufacture, or use of dugs (controlled substances).
- Public Crime:
  - Liquor Law Violation, Public Peace Violation, Weapons Violation, Interference with Public Officer, Concealed Carry License Violation, Obscenity, Public Indecency: These are offenses that disrupt public order, safety, and morals.
- Property Related Crime:
  - Criminal Damage, Criminal Trespass: These involve unlawful damage to or intrusion on someone else's property.
- Vulnerable Populations related Crime:
  - Offense Involving Children, Human Trafficking: These crimes are targeted at vulnerable groups, particularly children and victims of human trafficking, often involving exploitation or harm.
- Miscellaneous Crimes:
  - Other Offense, Ritualism, Non-Criminal, Non-Criminal (Subject Specified), Non - Criminal: This is a diverse category encompassing various offenses that don't neatly fit into the other categories. 'Non-criminal' may include acts that are not legally crimes but were still recorded for documentation purposes.

In [15]:
```
def categorize_crime(crime_type):
    economically_motivated = ['THEFT', 'BURGLARY', 'MOTOR VEHICLE THEFT', 'ROBBERY'
                              'DECEPTIVE PRACTICE', 'GAMBLING', 'PROSTITUTION', 'AR
    violent = ['BATTERY', 'ASSAULT', 'CRIM SEXUAL ASSAULT', 'CRIMINAL SEXUAL ASSAUL
               'HOMICIDE', 'KIDNAPPING', 'INTIMIDATION', 'SEX OFFENSE', 'STALKING']
    drug_related = ['NARCOTICS', 'OTHER NARCOTIC VIOLATION']
    public_crime = ['LIQUOR LAW VIOLATION', 'PUBLIC PEACE VIOLATION', 'WEAPONS VIOL
                    'INTERFERENCE WITH PUBLIC OFFICER', 'CONCEALED CARRY LICENSE VI
                    'OBSCENITY', 'PUBLIC INDECENCY']
    property_related = ['CRIMINAL DAMAGE', 'CRIMINAL TRESPASS']
    vulnerable_populations = ['OFFENSE INVOLVING CHILDREN', 'HUMAN TRAFFICKING']
    miscellaneous = ['OTHER OFFENSE', 'RITUALISM', 'NON-CRIMINAL',
                     'NON-CRIMINAL (SUBJECT SPECIFIED)', 'NON - CRIMINAL']

    if crime_type in economically_motivated:
        return "Economically Motivated"
    elif crime_type in violent:
        return "Violent"
    elif crime_type in drug_related:
        return "Drug-Related"
    elif crime_type in public_crime:
        return "Public Crime"
    elif crime_type in property_related:
        return "Property-Related"
    elif crime_type in vulnerable_populations:
        return "Vulnerable Populations"
    elif crime_type in miscellaneous:
        return "Miscellaneous"
    else:
        return "Other"

crimes_1022_small['Category'] = crimes_1022_small['Primary Type'].apply(categorize_

crimes_1022_small.head(3)
crimes_1022_small.tail(3)
```

Out[15]:

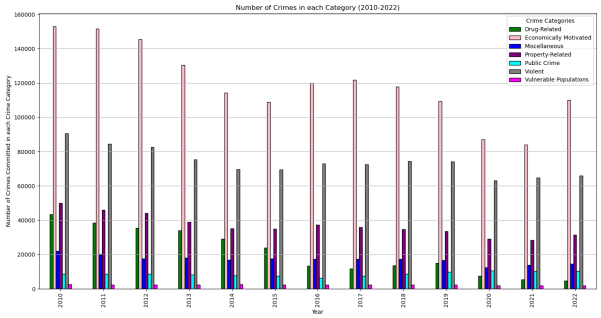
	ID	Case Number	Date	Block	Primary Type	Description	Arrest	Domestic	D
7918795	12118237	JD311791	2020-07-27 15:02:00	033XX W POLK ST	BATTERY	DOMESTIC BATTERY SIMPLE	False	True	
7918796	12142591	JD340297	2020-08-14 15:00:00	023XX W ROSEMONT AVE	MOTOR VEHICLE THEFT	AUTOMOBILE	False	False	
7918797	12002171	JD177406	2020-03-06 14:00:00	033XX N LAKEWOOD AVE	DECEPTIVE PRACTICE	CREDIT CARD FRAUD	False	False	

In [16]:
```
crimes_1022_small.to_csv('crimes_1022_small.csv', index=False)
```

3.1 Number of Crimes Per Category:

This Graph displays the number of crimes in each of the newly created categories from 2010-2022 to better visualize the count fluctuations in the categories over time.

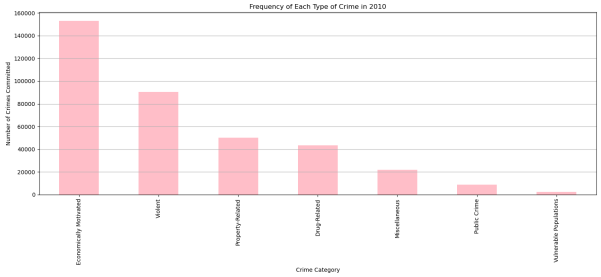
In [17]:
```
grouped_categories = crimes_1022_small.groupby(['Year', 'Category']).size().unstack

# Plot
fig, ax = plt.subplots(figsize=(15, 8))
grouped_categories.plot(kind='bar', stacked=False, ax=ax, color= ['green', 'pink',
plt.title('Number of Crimes in each Category (2010-2022)')
plt.xlabel('Year')
plt.ylabel('Number of Crimes Committed in each Crime Category')
plt.grid(axis='y')
plt.tight_layout()
plt.legend(title='Crime Categories')
plt.show()
```

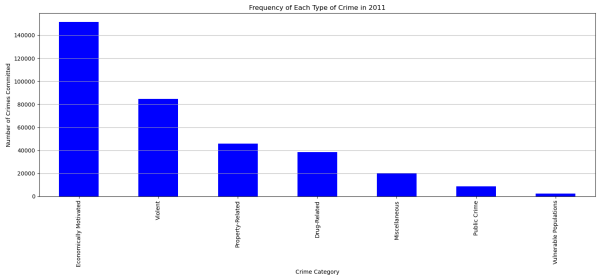


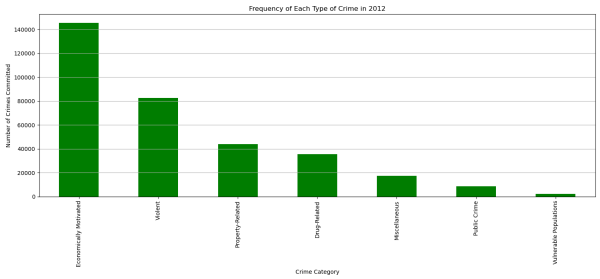
4. Breakdown of Number of Crimes Per Category

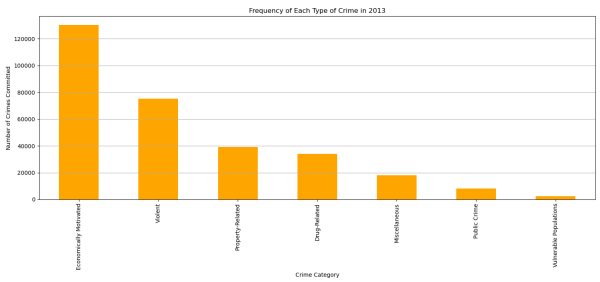
This is another way to display the number of crimes per category over time, by showing each year from 2010-2022 individually. This makes it easier to see which categories of crimes were committed more often in each year.

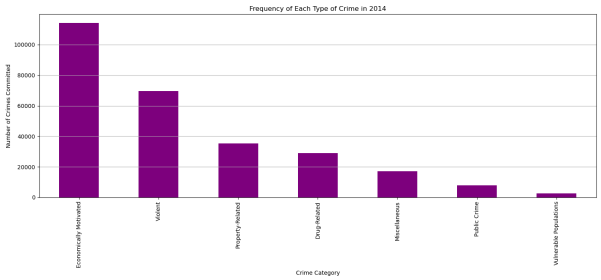
In [18]:
```
colors = ['pink', 'blue', 'green', 'orange', 'purple', 'cyan', 'yellow', 'red', 'br

for year, color in zip(range(2010, 2023), colors):
    crimes_year = crimes_1022_small[crimes_1022_small['Year'] == year]


    crime_counts_year = crimes_year['Category'].value_counts()

    crime_counts_year.plot(kind='bar', figsize=(15,7), color=color)
    plt.title(f'Frequency of Each Type of Crime in {year}')
    plt.xlabel('Crime Category')
    plt.ylabel('Number of Crimes Committed')
    plt.grid(axis='y')
    plt.tight_layout()
    plt.show()
```

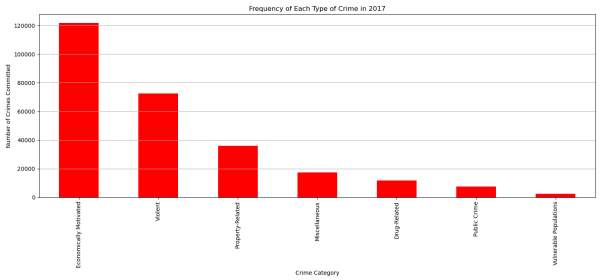


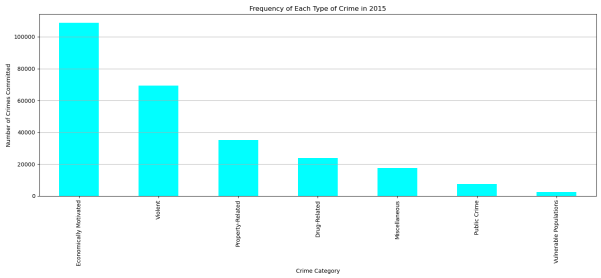


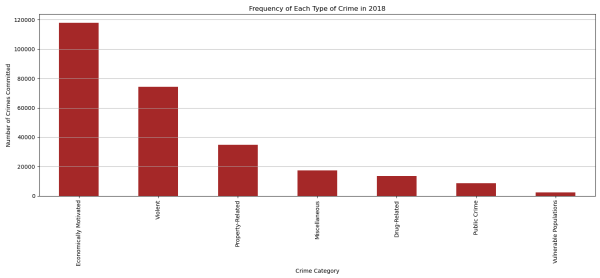


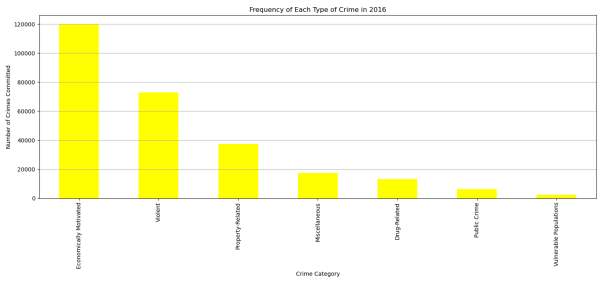


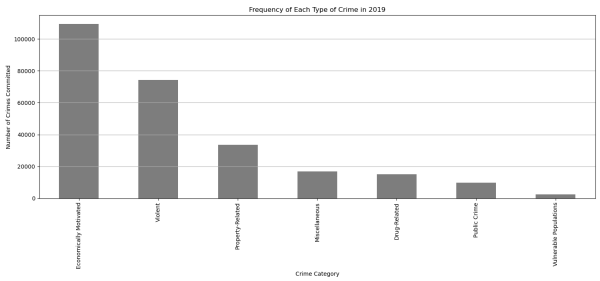


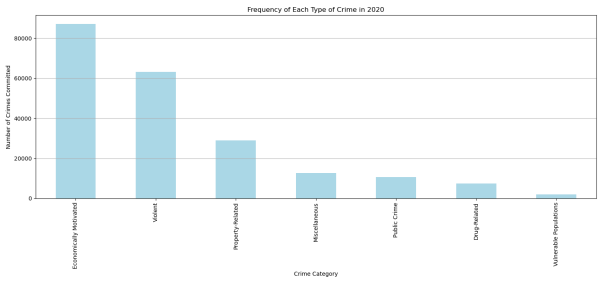


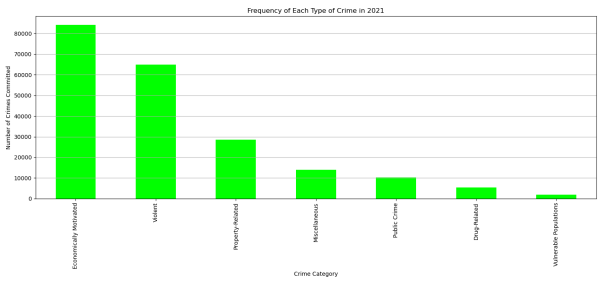


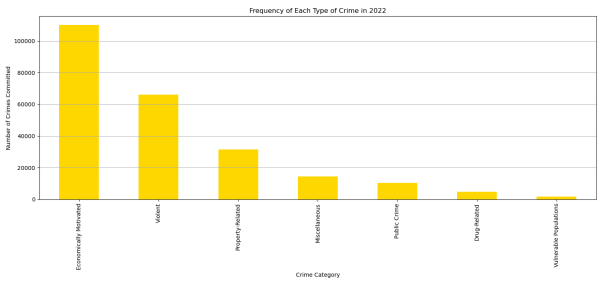












1.7 Web Scraping: Chicago Cost of Living Data

Accessed via https://www.numbeo.com/cost-of-living/city-history/in/Chicago

Now that I am happy with the chicago crimes data set, I will use webscraping to derive a Chicago cost of living data set from the site listed above.

In [20]:
```
URL = "https://www.numbeo.com/cost-of-living/city-history/in/Chicago"
response = requests.get(URL)

soup = BeautifulSoup(response.content, 'html.parser')

title = soup.title.string
print(title)
```

```
Historical Prices in Chicago, IL
```

The Numbeo website is structured with the use of tables. So, I am able to use beautiful soup to retrieve the tables off of the website. By right clicking the table on Numbeo that I want to retrieve and clicking on inspect, I can find the table name and then insert that name into the id = portion of code. In this case, numbeo is set up to very easily webscrape all of the tables as they are called tier_1 etc. I will take each table and assign it its own data frame name. I will then combine data frames from the same category until I have data frames with the following information: Average Cost of a Meal in a Resturaunt, Average cost of drinks, average cost of grocery store items, average cost of a shopping expenditures, average cost of leisure activities, average cost of transport, average rent costs, average housing purchase costs, and average cost of utilities. Then, I will combine all of these into a large data set, but I will still have the smaller ones to play around with during exploratory analysis if needed.

In [21]:
```
#This code template will be repeated until all of the tables have been webscraped c
rest_table = soup.find('table', id='tier_1')

headers_rest = [header.text for header in rest_table.find_all('th')]

rows_rest = rest_table.find_all('tr')
table_data = []
for row in rows_rest:
    row_rest_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_rest_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_rest = pd.DataFrame(table_data, columns=headers_rest)

print(df_rest.columns)
```

```
Index(['Year', 'Meal, Inexpensive Restaurant',
       'Meal for 2 People, Mid-range Restaurant, Three-course',
       'McMeal at McDonalds (or Equivalent Combo Meal)'],
      dtype='object')
```

After looking at this table, I realized that I want to include an average column as this will make analysis simpler later on. So, I will convert the numbers in the table to the same format and then I will add a column to the table for the average of the other columns (excluding the year column). I will most likely have to do this for every table that I web scrape.

In [22]:
```
df_rest['Meal, Inexpensive Restaurant'] = pd.to_numeric(df_rest['Meal, Inexpensive
df_rest['Meal for 2 People, Mid-range Restaurant, Three-course'] = pd.to_numeric(df
df_rest['McMeal at McDonalds (or Equivalent Combo Meal)'] = pd.to_numeric(df_rest['

df_rest['Average Resturaunt Meal Price'] = df_rest[['Meal, Inexpensive Restaurant',
                                                    'Meal for 2 People, Mid-range Restaurant,
                                                    'McMeal at McDonalds (or Equivalent Combo
#df_rest = df_rest.drop('Average Meal Price', axis=1)
print(df_rest)
```

```
    Year  Meal, Inexpensive Restaurant  \
0   2010                         18.00
1   2011                         10.00
2   2012                         10.00
3   2013                         10.00
4   2014                         11.00
5   2015                         14.50
6   2016                         15.00
7   2017                         13.75
8   2018                         15.00
9   2019                         15.00
10  2020                         15.00
11  2021                         18.00
12  2022                         20.00

    Meal for 2 People, Mid-range Restaurant, Three-course  \
0                                                35.0
1                                                40.0
2                                                60.0
3                                                55.0
4                                                60.0
5                                                65.0
6                                                60.0
7                                                60.0
8                                                67.5
9                                                65.0
10                                               75.0
11                                               82.5
12                                               80.0

    McMeal at McDonalds (or Equivalent Combo Meal)  \
0                                             5.00
1                                             7.00
2                                             6.50
3                                             6.40
4                                             6.50
5                                             7.00
6                                             7.25
7                                             7.00
8                                             7.00
9                                             8.00
10                                            8.00
11                                            8.00
12                                           10.00

    Average Resturaunt Meal Price
0                       19.333333
1                       19.000000
2                       25.500000
3                       23.800000
4                       25.833333
5                       28.833333
6                       27.416667
7                       26.916667
8                       29.833333
9                       29.333333
10                      32.666667
11                      36.166667
12                      36.666667
```

In [23]:
```
drink_table= soup.find('table', id='tier_2')
headers_drink = [header.text for header in drink_table.find_all('th')]

rows_drink = drink_table.find_all('tr')
```

```
    Year  Domestic_Beer_0.5_liter_draught  Imported_Beer_0.33_liter_bottle  \
0   2010                              4.0                             6.00
1   2011                              3.0                             5.00
2   2012                              5.0                             5.76
3   2013                              4.0                             5.25
4   2014                              4.0                             6.00
5   2015                              5.0                             6.00
6   2016                              5.0                             6.00
7   2017                              5.0                             7.00
8   2018                              5.0                             6.50
9   2019                              5.0                             7.00
10  2020                              5.5                             7.00
11  2021                              5.0                             7.00
12  2022                              6.5                             7.50

    Coke/Pepsi_0.33_liter_bottle  Water_0.33_liter_bottle_  \
0                           1.69                      1.12
1                           1.56                      1.29
2                           1.86                      1.50
3                           1.49                      1.23
4                           1.76                      1.56
5                           1.77                      1.70
6                           1.86                      1.64
7                           1.99                      1.73
8                           1.82                      1.66
9                           1.97                      1.77
10                          2.03                      1.57
11                          2.05                      1.81
12                          2.51                      2.10

    Cappuccino_regular  Average Drink Price
0                  NaN               3.2025
1                 3.42               2.8540
2                 3.79               3.5820
3                 3.60               3.1140
4                 3.82               3.4280
5                 3.83               3.6600
6                 3.91               3.6820
7                 4.05               3.9540
8                 4.19               3.8340
9                 4.17               3.9820
10                4.05               4.0300
11                4.43               4.0580
12                5.24               4.7700
```

Now, I will merge the two previously completed data frames for resturaunts and drinks to create one table with both items of similiar meaning. This new large data frame will also have the average columns added previously to the smaller data frames.

In [27]:
```
df_allfooddrink = pd.merge(df_rest, df_drink, on='Year', suffixes=('_rest', '_drink
print(df_allfooddrink)
```

```
table_data = []
for row in rows_drink:
    row_drink_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_drink_data)

table_data = [row for row in table_data if len(row) > 0]
table_data = sorted(table_data, key=lambda x: x[0])
df_drink = pd.DataFrame(table_data, columns=headers_drink)

print(df_drink.columns)
```

```
Index(['Year', 'Domestic Beer (0.5 liter draught)',
       'Imported Beer (0.33 liter bottle)', 'Coke/Pepsi (0.33 liter bottle)',
       'Water (0.33 liter bottle) ', 'Cappuccino (regular)'],
      dtype='object')
```

In [24]:
```
df_drink.columns=df_drink.columns.str.replace(' ', '_')
df_drink.columns= df_drink.columns.str.replace('(', '')
df_drink.columns= df_drink.columns.str.replace(')', '')
```

In [25]:
```
print(df_drink.columns)
```

```
Index(['Year', 'Domestic_Beer_0.5_liter_draught',
       'Imported_Beer_0.33_liter_bottle', 'Coke/Pepsi_0.33_liter_bottle',
       'Water_0.33_liter_bottle_', 'Cappuccino_regular'],
      dtype='object')
```

In [26]:
```
#Calculating the average and adding it as a column to the data frame
df_drink['Domestic_Beer_0.5_liter_draught'] = pd.to_numeric(df_drink['Domestic_Beer
df_drink['Imported_Beer_0.33_liter_bottle'] = pd.to_numeric(df_drink['Imported_Beer
df_drink['Coke/Pepsi_0.33_liter_bottle'] = pd.to_numeric(df_drink['Coke/Pepsi_0.33_
df_drink['Water_0.33_liter_bottle_'] = pd.to_numeric(df_drink['Water_0.33_liter_bot
df_drink['Cappuccino_regular'] = pd.to_numeric(df_drink['Cappuccino_regular'], errc

df_drink['Average Drink Price'] = df_drink[['Domestic_Beer_0.5_liter_draught',
                                            'Imported_Beer_0.33_liter_bottle',
                                            'Coke/Pepsi_0.33_liter_bottle',
                                            'Water_0.33_liter_bottle_',
                                            'Cappuccino_regular']].mean(axis=1)

print(df_drink)
```

```
    Year  Meal, Inexpensive Restaurant  \
0   2010                         18.00
1   2011                         10.00
2   2012                         10.00
3   2013                         10.00
4   2014                         11.00
5   2015                         14.50
6   2016                         15.00
7   2017                         13.75
8   2018                         15.00
9   2019                         15.00
10  2020                         15.00
11  2021                         18.00
12  2022                         20.00

    Meal for 2 People, Mid-range Restaurant, Three-course  \
0                                                35.0
1                                                40.0
2                                                60.0
3                                                55.0
4                                                60.0
5                                                65.0
6                                                60.0
7                                                60.0
8                                                67.5
9                                                65.0
10                                               75.0
11                                               82.5
12                                               80.0

    McMeal at McDonalds (or Equivalent Combo Meal)  \
0                                             5.00
1                                             7.00
2                                             6.50
3                                             6.40
4                                             6.50
5                                             7.00
6                                             7.25
7                                             7.00
8                                             7.00
9                                             8.00
10                                            8.00
11                                            8.00
12                                           10.00

    Average Resturaunt Meal Price  Domestic_Beer_0.5_liter_draught  \
0                       19.333333                              4.0
1                       19.000000                              3.0
2                       25.500000                              5.0
3                       23.800000                              4.0
4                       25.833333                              4.0
5                       28.833333                              5.0
6                       27.416667                              5.0
7                       26.916667                              5.0
8                       29.833333                              5.0
9                       29.333333                              5.0
10                      32.666667                              5.5
11                      36.166667                              5.0
12                      36.666667                              6.5

    Imported_Beer_0.33_liter_bottle  Coke/Pepsi_0.33_liter_bottle  \
0                              6.00                          1.69
1                              5.00                          1.56
2                              5.76                          1.86
```

```
3                            5.25                          1.49
4                            6.00                          1.76
5                            6.00                          1.77
6                            6.00                          1.86
7                            7.00                          1.99
8                            6.50                          1.82
9                            7.00                          1.97
10                           7.00                          2.03
11                           7.00                          2.05
12                           7.50                          2.51

    Water_0.33_liter_bottle_  Cappuccino_regular  Average Drink Price
0                       1.12                 NaN               3.2025
1                       1.29                3.42               2.8540
2                       1.50                3.79               3.5820
3                       1.23                3.60               3.1140
4                       1.56                3.82               3.4280
5                       1.70                3.83               3.6600
6                       1.64                3.91               3.6820
7                       1.73                4.05               3.9540
8                       1.66                4.19               3.8340
9                       1.77                4.17               3.9820
10                      1.57                4.05               4.0300
11                      1.81                4.43               4.0580
12                      2.10                5.24               4.7700
```

In [28]:
```
print(df_allfooddrink.isna().sum())
```

```
Year                                                  0
Meal, Inexpensive Restaurant                          0
Meal for 2 People, Mid-range Restaurant, Three-course 0
McMeal at McDonalds (or Equivalent Combo Meal)        0
Average Resturaunt Meal Price                         0
Domestic_Beer_0.5_liter_draught                       0
Imported_Beer_0.33_liter_bottle                       0
Coke/Pepsi_0.33_liter_bottle                          0
Water_0.33_liter_bottle_                              0
Cappuccino_regular                                    1
Average Drink Price                                   0
dtype: int64
```

I noticed that there is one value missing in the Cappuccino_regular column. Thinking of ways to fix this, I looked up the inflation rate in 2011 and 2010. I then used the formula: Price in 2011 divided by 1 + the inflation rate of 2010 to 2011 to find the approximate price of a cappucino in 2010. This logic will be used later on if other missing values are found. In this case, a cappucino in 2010 would cost approximately $3.32.

In [29]:
```
df_allfooddrink['Cappuccino_regular'].fillna(3.32, limit=1, inplace=True)
print(df_allfooddrink['Cappuccino_regular'])
```

```
0     3.32
1     3.42
2     3.79
3     3.60
4     3.82
5     3.83
6     3.91
7     4.05
8     4.19
9     4.17
10    4.05
11    4.43
12    5.24
Name: Cappuccino_regular, dtype: float64
```

In [30]:
```
#Now that this data frame is complete, I will download it to not lose my work if th
df_allfooddrink.to_csv('allfooddrink_data.csv', index=False)
```

Onto the next table to be web scraped.

In [31]:
```
market_table= soup.find('table', id='tier_3')
headers_market = [header.text for header in market_table.find_all('th')]

rows_market = market_table.find_all('tr')
table_data = []
for row in rows_market:
    row_market_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_market_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_market = pd.DataFrame(table_data, columns=headers_market)

print(df_market.columns)
```

```
Index(['Year', 'Milk (regular), (1 liter)', 'Loaf of Fresh White Bread (500g)',
       'Eggs (regular) (12)', 'Water (1.5 liter bottle)',
       'Domestic Beer (0.5 liter bottle)',
       'Imported Beer (0.33 liter bottle)'],
      dtype='object')
```

*I realized that domestic beer, imported beer, and bottled water prices were already explored above, so I removed these columns from the market data frame.

In [32]:
```
df_market = df_market.drop(['Domestic Beer (0.5 liter bottle)', 'Imported Beer (0.3

print(df_market.columns)
```

```
Index(['Year', 'Milk (regular), (1 liter)', 'Loaf of Fresh White Bread (500g)',
       'Eggs (regular) (12)'],
      dtype='object')
```

In [33]:
```
produce_table = soup.find('table', id='tier_4')
headers_produce = [header.text for header in produce_table.find_all('th')]

rows_produce = produce_table.find_all('tr')
table_data = []
for row in rows_produce:
    row_produce_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_produce_data)

table_data = [row for row in table_data if len(row) > 0]
table_data = sorted(table_data, key=lambda x: x[0])
```

```
df_produce = pd.DataFrame(table_data, columns=headers_produce)

print(df_produce)
```

```
    Year Apples (1kg) Oranges (1kg) Potato (1kg) Lettuce (1 head)  \
0   2010           -             -            -                 -
1   2011        2.21          2.00         1.42              0.80
2   2012        2.76          2.41         3.37              1.16
3   2013        4.14          2.65         1.80              1.34
4   2014        3.10          2.82         1.31              1.38
5   2015        4.31          3.92         2.52              1.60
6   2016        3.88          3.83         2.54              1.31
7   2017        3.65          4.02         1.93              1.46
8   2018        5.16          4.13         2.20              1.44
9   2019        4.54          3.69         2.42              1.80
10  2020        4.70          4.08         3.71              1.82
11  2021        3.93          2.82         1.90              1.61
12  2022        5.24          4.80         2.64              1.89

   Rice (white), (1kg) Tomato (1kg) Banana (1kg) Onion (1kg)
0                    -            -            -          -
1                    -            -            -          -
2                 1.43         3.00            -          -
3                 2.27         4.36            -          -
4                 2.22         4.31            -          -
5                 3.38         3.91         1.70       2.98
6                 5.21         3.96         1.29       2.56
7                 4.47         3.07         1.77       1.81
8                 4.16         3.47         1.79       2.45
9                    -         4.54         1.46       2.64
10                4.39         5.00         1.58       2.77
11                3.52         3.26         1.57       2.07
12                4.17         4.87         1.72       2.99
```

As you can see, I am missing quite a few values from the earlier years. So, I will apply the same logic as before and use the inflation rates to fill in this missing data. But first, I will gather all relevant data from Numbeo to include in the table.

In [34]:
```
produce2_table = soup.find('table', id='tier_5')
headers_produce2 = [header.text for header in produce2_table.find_all('th')]

rows_produce2 = produce2_table.find_all('tr')
table_data = []
for row in rows_produce2:
    row_produce2_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_produce2_data)

table_data = [row for row in table_data if len(row) > 0]
table_data = sorted(table_data, key=lambda x: x[0])
df_produce2 = pd.DataFrame(table_data, columns=headers_produce2)

print(df_produce2)
```

```
    Year Local Cheese (1kg) Bottle of Wine (Mid-Range)  \
0   2010             13.21                       15.00
1   2011              7.26                        8.00
2   2012              9.55                       13.50
3   2013              8.53                       12.00
4   2014              6.38                       10.00
5   2015             14.08                       12.00
6   2016              9.75                       14.50
7   2017             14.90                       15.00
8   2018             10.29                       13.00
9   2019             12.28                       14.00
10  2020             11.72                       12.00
11  2021             10.05                       10.00
12  2022             18.12                       15.00

   Cigarettes 20 Pack (Marlboro) Chicken Fillets (1kg)  \
0                           7.00                 12.10
1                           8.50                  5.63
2                          10.00                  7.57
3                           9.00                  8.81
4                          11.25                  9.83
5                          12.00                 11.80
6                          12.00                  9.91
7                          12.00                 10.29
8                          13.00                 10.83
9                              -                 10.14
10                         15.00                  9.92
11                         15.00                  8.52
12                         16.50                 12.81

   Beef Round (1kg) (or Equivalent Back Leg Red Meat)
0                                                   -
1                                                   -
2                                                   -
3                                                   -
4                                                   -
5                                               13.22
6                                               12.97
7                                               11.41
8                                               13.63
9                                               14.55
10                                              11.63
11                                              12.36
12                                              17.21
```

In [35]:
```
df_combined_produce = pd.merge(df_produce, df_produce2, on='Year', how='outer')
df_allmarket = pd.merge(df_combined_produce, df_market, on='Year', how='outer')
print(df_allmarket.columns)
```

```
Index(['Year', 'Apples (1kg)', 'Oranges (1kg)', 'Potato (1kg)',
       'Lettuce (1 head)', 'Rice (white), (1kg)', 'Tomato (1kg)',
       'Banana (1kg)', 'Onion (1kg)', 'Local Cheese (1kg)',
       'Bottle of Wine (Mid-Range)', 'Cigarettes 20 Pack (Marlboro)',
       'Chicken Fillets (1kg)',
       'Beef Round (1kg) (or Equivalent Back Leg Red Meat)',
       'Milk (regular), (1 liter)', 'Loaf of Fresh White Bread (500g)',
       'Eggs (regular) (12)'],
      dtype='object')
```

In [36]:
```
df_allmarket
```

Out[36]:

	Year	Apples (1kg)	Oranges (1kg)	Potato (1kg)	Lettuce (1 head)	Rice (white), (1kg)	Tomato (1kg)	Banana (1kg)	Onion (1kg)	Local Cheese (1kg)	Bottle of Wine (Mid-Range)
0	2010	-	-	-	-	-	-	-	-	13.21	15.00
1	2011	2.21	2.00	1.42	0.80	-	-	-	-	7.26	8.00
2	2012	2.76	2.41	3.37	1.16	1.43	3.00	-	-	9.55	13.50
3	2013	4.14	2.65	1.80	1.34	2.27	4.36	-	-	8.53	12.00
4	2014	3.10	2.82	1.31	1.38	2.22	4.31	-	-	6.38	10.00
5	2015	4.31	3.92	2.52	1.60	3.38	3.91	1.70	2.98	14.08	12.00
6	2016	3.88	3.83	2.54	1.31	5.21	3.96	1.29	2.56	9.75	14.50
7	2017	3.65	4.02	1.93	1.46	4.47	3.07	1.77	1.81	14.90	15.00
8	2018	5.16	4.13	2.20	1.44	4.16	3.47	1.79	2.45	10.29	13.00
9	2019	4.54	3.69	2.42	1.80	-	4.54	1.46	2.64	12.28	14.00
10	2020	4.70	4.08	3.71	1.82	4.39	5.00	1.58	2.77	11.72	12.00
11	2021	3.93	2.82	1.90	1.61	3.52	3.26	1.57	2.07	10.05	10.00
12	2022	5.24	4.80	2.64	1.89	4.17	4.87	1.72	2.99	18.12	15.00

There's quite a few missing values here, so I will need to webscrape again to source an inflation rate table for the years 2010-2022 in order to fill in this missing data with approximate values. I found a good table with this data at www.usinflationcalculator.com/inflation/current-inflation-rates/. So, I will use this to get the average inflation rate for 2010-2022.

In [37]:
```
url = 'https://www.usinflationcalculator.com/inflation/current-inflation-rates/'

response = requests.get(url)

if response.status_code == 200:
    tables = pd.read_html(response.text)
    df_inflation = tables[0]
    print(df_inflation)
else:
    print("Failed to retrieve the webpage")
```

```
      0     1    2    3    4    5    6    7    8    9   \
0  Year   Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep
1  2023   6.4  6.0  5.0  4.9  4.0  3.0  3.2  3.7  3.7
2  2022   7.5  7.9  8.5  8.3  8.6  9.1  8.5  8.3  8.2
3  2021   1.4  1.7  2.6  4.2  5.0  5.4  5.4  5.3  5.4
4  2020   2.5  2.3  1.5  0.3  0.1  0.6  1.0  1.3  1.4
5  2019   1.6  1.5  1.9  2.0  1.8  1.6  1.8  1.7  1.7
6  2018   2.1  2.2  2.4  2.5  2.8  2.9  2.9  2.7  2.3
7  2017   2.5  2.7  2.4  2.2  1.9  1.6  1.7  1.9  2.2
8  2016   1.4  1.0  0.9  1.1  1.0  1.0  0.8  1.1  1.5
9  2015  -0.1  0.0 -0.1 -0.2  0.0  0.1  0.2  0.2  0.0
10 2014   1.6  1.1  1.5  2.0  2.1  2.1  2.0  1.7  1.7
11 2013   1.6  2.0  1.5  1.1  1.4  1.8  2.0  1.5  1.2
12 2012   2.9  2.9  2.7  2.3  1.7  1.7  1.4  1.7  2.0
13 2011   1.6  2.1  2.7  3.2  3.6  3.6  3.6  3.8  3.9
14 2010   2.6  2.1  2.3  2.2  2.0  1.1  1.2  1.1  1.1
15 2009     0  0.2 -0.4 -0.7 -1.3 -1.4 -2.1 -1.5 -1.3
16 2008   4.3  4.0  4.0  3.9  4.2  5.0  5.6  5.4  4.9
17 2007   2.1  2.4  2.8  2.6  2.7  2.7  2.4  2.0  2.8
18 2006   4.0  3.6  3.4  3.5  4.2  4.3  4.1  3.8  2.1
19 2005   3.0  3.0  3.1  3.5  2.8  2.5  3.2  3.6  4.7
20 2004   1.9  1.7  1.7  2.3  3.1  3.3  3.0  2.7  2.5
21 2003   2.6  3.0  3.0  2.2  2.1  2.1  2.1  2.2  2.3
22 2002   1.1  1.1  1.5  1.6  1.2  1.1  1.5  1.8  1.5
23 2001   3.7  3.5  2.9  3.3  3.6  3.2  2.7  2.7  2.6
24 2000   2.7  3.2  3.8  3.1  3.2  3.7  3.7  3.4  3.5

                10   11   12   13
0               Oct  Nov  Dec  Ave
1   Avail.  Nov.  14  NaN  NaN  NaN
2                7.7  7.1  6.5  8.0
3                6.2  6.8  7.0  4.7
4                1.2  1.2  1.4  1.2
5                1.8  2.1  2.3  1.8
6                2.5  2.2  1.9  2.4
7                2.0  2.2  2.1  2.1
8                1.6  1.7  2.1  1.3
9                0.2  0.5  0.7  0.1
10               1.7  1.3  0.8  1.6
11               1.0  1.2  1.5  1.5
12               2.2  1.8  1.7  2.1
13               3.5  3.4  3.0  3.2
14               1.2  1.1  1.5  1.6
15              -0.2  1.8  2.7 -0.4
16               3.7  1.1  0.1  3.8
17               3.5  4.3  4.1  2.8
18               1.3  2.0  2.5  3.2
19               4.3  3.5  3.4  3.4
20               3.2  3.5  3.3  2.7
21               2.0  1.8  1.9  2.3
22               2.0  2.2  2.4  1.6
23               2.1  1.9  1.6  2.8
24               3.4  3.4  3.4  3.4
```

In [38]:
```
df_inflation = df_inflation.iloc[:, [0, 13]]

print(df_inflation)
```

```
      0     13
0   Year   Ave
1   2023   NaN
2   2022   8.0
3   2021   4.7
4   2020   1.2
5   2019   1.8
6   2018   2.4
7   2017   2.1
8   2016   1.3
9   2015   0.1
10  2014   1.6
11  2013   1.5
12  2012   2.1
13  2011   3.2
14  2010   1.6
15  2009  -0.4
16  2008   3.8
17  2007   2.8
18  2006   3.2
19  2005   3.4
20  2004   2.7
21  2003   2.3
22  2002   1.6
23  2001   2.8
24  2000   3.4
```

In [39]:
```
df_inflation = [
    ["Year", "Ave"],
    [2023, None],
    [2022, 8.0],
    [2021, 4.7],
    [2020, 1.2],
    [2019, 1.8],
    [2018, 2.4],
    [2017, 2.1],
    [2016, 1.3],
    [2015, 0.1],
    [2014, 1.6],
    [2013, 1.5],
    [2012, 2.1],
    [2011, 3.2],
    [2010, 1.6],
    [2009, -0.4],
    [2008, 3.8],
    [2007, 2.8],
    [2006, 3.2],
    [2005, 3.4],
    [2004, 2.7],
    [2003, 2.3],
    [2002, 1.6],
    [2001, 2.8],
    [2000, 3.4]
                ]

df_inflation = pd.DataFrame(df_inflation[1:], columns=df_inflation[0])

df_inflation['Year'] = pd.to_numeric(df_inflation['Year'])

df_inflation1022 = df_inflation[(df_inflation['Year'] >= 2010) & (df_inflation['Yea

print(df_inflation1022)
#Now I have the years that I need the inflation rate for and it is in a readable ta
```

```
   Year  Ave
1  2022  8.0
2  2021  4.7
3  2020  1.2
4  2019  1.8
5  2018  2.4
6  2017  2.1
7  2016  1.3
8  2015  0.1
9  2014  1.6
10 2013  1.5
11 2012  2.1
12 2011  3.2
13 2010  1.6
```

In [40]:
```
df_inflation1022.to_csv('inflation_2010_2022.csv', index=False)
#Now that I am happy with the inflation rate table, I have saved it to a csv file f
```

The below code did not work. I figured out that this was because I had several missing values in a row and the code did not account for this as it was filling forwards instead of backwards. So, I figured out a different solution. There was quite a bit of trial and error for this, so I wanted to display one of the incorrect tries to show how I overcame this challenge.

```
inflation_dict = df_inflation1022.set_index('Year')['Ave'].to_dict()

    # I'll use this code for all columns in df_allmarket

for column in df_allmarket.columns: if column != 'Year': # well, all columns except the year column for i in range(len(df_allmarket)): if pd.isna(df_allmarket[column].iloc[i]):

                # if there is a missing value, I'll find the nearest
    value not missing from the next year to use
                for j in range(i - 1, -1, -1):
                    if not pd.isna(df_allmarket[column].iloc[j]):
                        prev_year = df_allmarket['Year'].iloc[j]
                        prev_value = df_allmarket[column].iloc[j]
                        curr_year = df_allmarket['Year'].iloc[i]

                        #This will be calculated using the same formula
    as above for the single cappucino entry
                        while prev_year < curr_year:
                            inflation_rate =
    inflation_dict.get(prev_year + 1, 0) / 100
                            prev_value *= (1 + inflation_rate)
                            prev_year += 1

                        df_allmarket[column].iloc[i] = prev_value
                        break
```

In [41]:
```
inflation_dict = df_inflation1022.set_index('Year')['Ave'].to_dict()
#for all of the columns in df_allmarket, I am skipping the year column, I will find
for column in df_allmarket.columns:
    if column != 'Year':  # Skip 'Year' column
        for i in range(len(df_allmarket)):
            if df_allmarket[column].iloc[i] == '-':
                found = False
                for j in range(i + 1, len(df_allmarket)):
                    if df_allmarket[column].iloc[j] != '-':
```

```
                        found = True
                        next_year = int(df_allmarket['Year'].iloc[j])
                        next_value = float(df_allmarket[column].iloc[j])
                        break
#when missing values are found, I will use the next available year's data and infla
                if found:
                    for k in range(j-1, i-1, -1):
                        curr_year = int(df_allmarket['Year'].iloc[k])
                        next_year = int(df_allmarket['Year'].iloc[k + 1])

                        inflation_rate = inflation_dict.get(next_year, 0) / 100

                        next_value /= (1 + inflation_rate)

                        df_allmarket[column].iloc[k] = next_value
#Then, I can use the newly calculated values to fill in other missing values and so
```

In [42]:
```
#When this finally worked and all the data was backfilled correctly, I was thrilled

print(df_allmarket)
```

```
    Year Apples (1kg) Oranges (1kg) Potato (1kg) Lettuce (1 head)  \
0   2010     2.141473      1.937984     1.375969          0.775194
1   2011         2.21          2.00         1.42              0.80
2   2012         2.76          2.41         3.37              1.16
3   2013         4.14          2.65         1.80              1.34
4   2014         3.10          2.82         1.31              1.38
5   2015         4.31          3.92         2.52              1.60
6   2016         3.88          3.83         2.54              1.31
7   2017         3.65          4.02         1.93              1.46
8   2018         5.16          4.13         2.20              1.44
9   2019         4.54          3.69         2.42              1.80
10  2020         4.70          4.08         3.71              1.82
11  2021         3.93          2.82         1.90              1.61
12  2022         5.24          4.80         2.64              1.89

    Rice (white), (1kg) Tomato (1kg) Banana (1kg) Onion (1kg)  \
0              1.357159     2.847186     1.562966    2.739788
1              1.400588     2.938296     1.612981    2.827461
2                  1.43         3.00     1.646854    2.886838
3                  2.27         4.36     1.671557    2.930141
4                  2.22         4.31     1.698302    2.977023
5                  3.38         3.91         1.70        2.98
6                  5.21         3.96         1.29        2.56
7                  4.47         3.07         1.77        1.81
8                  4.16         3.47         1.79        2.45
9              4.337945         4.54         1.46        2.64
10                 4.39         5.00         1.58        2.77
11                 3.52         3.26         1.57        2.07
12                 4.17         4.87         1.72        2.99

    Local Cheese (1kg) Bottle of Wine (Mid-Range)  \
0                13.21                       15.00
1                 7.26                        8.00
2                 9.55                       13.50
3                 8.53                       12.00
4                 6.38                       10.00
5                14.08                       12.00
6                 9.75                       14.50
7                14.90                       15.00
8                10.29                       13.00
9                12.28                       14.00
10               11.72                       12.00
11               10.05                       10.00
12               18.12                       15.00

    Cigarettes 20 Pack (Marlboro) Chicken Fillets (1kg)  \
0                            7.00                 12.10
1                            8.50                  5.63
2                           10.00                  7.57
3                            9.00                  8.81
4                           11.25                  9.83
5                           12.00                 11.80
6                           12.00                  9.91
7                           12.00                 10.29
8                           13.00                 10.83
9                      14.822134                 10.14
10                          15.00                  9.92
11                          15.00                  8.52
12                          16.50                 12.81

    Beef Round (1kg) (or Equivalent Back Leg Red Meat)  \
0                                           12.154363
1                                           12.543302
2                                           12.806712
```

```
3                                       12.998812
4                                       13.206793
5                                           13.22
6                                           12.97
7                                           11.41
8                                           13.63
9                                           14.55
10                                          11.63
11                                          12.36
12                                          17.21

    Milk (regular), (1 liter) Loaf of Fresh White Bread (500g)  \
0                         1.59                              2.25
1                         0.95                              2.33
2                         0.88                              2.32
3                         0.84                              1.97
4                         1.04                              1.98
5                         0.85                              2.78
6                         0.78                              2.59
7                         0.83                              2.35
8                         0.75                              2.76
9                         0.88                              3.05
10                        0.78                              3.05
11                        0.80                              2.70
12                        0.93                              3.38

    Eggs (regular) (12)
0                  1.94
1                  1.82
2                  2.34
3                  2.11
4                  2.09
5                  2.93
6                  2.51
7                  2.01
8                  2.32
9                  2.30
10                 2.30
11                 2.14
12                 3.07
```

In [43]:
```
for col in df_allmarket.columns:
    if col != 'Year':
        df_allmarket[col] = pd.to_numeric(df_allmarket[col], errors='coerce')

df_allmarket['Total Average Cost of all Groceries'] = df_allmarket.drop('Year', axi

#print(df_allmarket)
```

In [44]:
```
df_allmarket.to_csv('df_allmarket.csv', index=False)
```

In [45]:
```
rent_table= soup.find('table', id='tier_6')
headers_rent = [header.text for header in rent_table.find_all('th')]

rows_rent = rent_table.find_all('tr')
table_data = []
for row in rows_rent:
    row_rent_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_rent_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_rent = pd.DataFrame(table_data, columns=headers_rent)
```

```
print(df_rent.columns)
```

```
Index(['Year', 'Apartment (1 bedroom) in City Centre',
       'Apartment (1 bedroom) Outside of Centre',
       'Apartment (3 bedrooms) in City Centre',
       'Apartment (3 bedrooms) Outside of Centre'],
      dtype='object')
```

In [46]:
```
rent_columns = ['Apartment (1 bedroom) in City Centre',
                'Apartment (1 bedroom) Outside of Centre',
                'Apartment (3 bedrooms) in City Centre',
                'Apartment (3 bedrooms) Outside of Centre']

for column in rent_columns:
    df_rent[column] = pd.to_numeric(df_rent[column], errors='coerce')

df_rent['Average Rent Price'] = df_rent[rent_columns].mean(axis=1)

print(df_rent)

missing_values = df_rent.isnull().sum()
print(missing_values)
#No missing values here, yay

df_rent.to_csv('df_rent.csv', index=False)
```

```
    Year  Apartment (1 bedroom) in City Centre  \
0   2010                                924.00
1   2011                               1416.67
2   2012                               1464.22
3   2013                               1433.93
4   2014                               1689.55
5   2015                               1553.70
6   2016                               1753.32
7   2017                               1782.28
8   2018                               1718.58
9   2019                               1934.04
10  2020                               1875.84
11  2021                               1782.90
12  2022                               2045.81

    Apartment (1 bedroom) Outside of Centre  \
0                                    822.00
1                                    881.25
2                                    910.00
3                                    973.18
4                                   1010.45
5                                   1021.95
6                                   1094.77
7                                   1124.15
8                                   1090.68
9                                   1256.51
10                                  1280.85
11                                  1217.50
12                                  1456.40

    Apartment (3 bedrooms) in City Centre  \
0                                 1945.00
1                                 2650.00
2                                 2920.00
3                                 2900.00
4                                 2800.00
5                                 3106.71
6                                 3273.67
7                                 3187.14
8                                 3129.77
9                                 3687.55
10                                3587.95
11                                3319.23
12                                4152.94

    Apartment (3 bedrooms) Outside of Centre  Average Rent Price
0                                    1512.50           1300.8750
1                                    1350.00           1574.4800
2                                    1606.25           1725.1175
3                                    2150.00           1864.2775
4                                    1778.17           1819.5425
5                                    1841.43           1880.9475
6                                    1904.22           2006.4950
7                                    2008.43           2025.5000
8                                    1915.20           1963.5575
9                                    2222.75           2275.2125
10                                   2216.53           2240.2925
11                                   2006.67           2081.5750
12                                   2600.00           2563.7875
Year                                       0
Apartment (1 bedroom) in City Centre       0
Apartment (1 bedroom) Outside of Centre    0
Apartment (3 bedrooms) in City Centre      0
Apartment (3 bedrooms) Outside of Centre   0
```

```
    Year  Price per Square Meter to Buy Apartment in City Centre  \
0   2010                                            1883.68
1   2011                                            2152.78
2   2012                                            2152.78
3   2013                                                NaN
4   2014                                            3354.75
5   2015                                            3408.04
6   2016                                            3267.14
7   2017                                            3257.04
8   2018                                            3762.85
9   2019                                            4351.41
10  2020                                            6186.71
11  2021                                            3632.82
12  2022                                                NaN

    Price per Square Meter to Buy Apartment Outside of Centre  \
0                                           1372.40
1                                           1345.49
2                                           1119.45
3                                               NaN
4                                               NaN
5                                           1811.03
6                                           1795.11
7                                           1892.12
8                                           2116.87
9                                           2031.03
10                                          2289.42
11                                          2389.59
12                                              NaN

    Average per Square Meter Buy Price
0                             1628.040
1                             1749.135
2                             1636.115
3                                  NaN
4                             3354.750
5                             2609.535
6                             2531.125
7                             2574.580
8                             2939.860
9                             3191.220
10                            4238.065
11                            3011.205
12                                 NaN
Year                                                          0
Price per Square Meter to Buy Apartment in City Centre        2
Price per Square Meter to Buy Apartment Outside of Centre     3
Average per Square Meter Buy Price                            2
dtype: int64
```

In [49]:
```
inflation_dict = df_inflation1022.set_index('Year')['Ave'].to_dict()

for column in df_buy.columns:
    if column != 'Year':
        for i in range(len(df_buy)):
            if pd.isna(df_buy[column].iloc[i]):
                found = False
                for j in range(i + 1, len(df_buy)):
                    if not pd.isna(df_buy[column].iloc[j]):
                        found = True
                        next_year = int(df_buy['Year'].iloc[j])
                        next_value = float(df_buy[column].iloc[j])
                        break
```

```
Average Rent Price                          0
dtype: int64
```

In [47]:
```
buy_table= soup.find('table', id='tier_7')
headers_buy = [header.text for header in buy_table.find_all('th')]

rows_buy = buy_table.find_all('tr')
table_data = []
for row in rows_buy:
    row_buy_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_buy_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_buy = pd.DataFrame(table_data, columns=headers_buy)

print(df_buy.columns)
```

```
Index(['Year', 'Price per Square Meter to Buy Apartment in City Centre',
       'Price per Square Meter to Buy Apartment Outside of Centre'],
      dtype='object')
```

In [48]:
```
buy_columns = ['Price per Square Meter to Buy Apartment in City Centre',
               'Price per Square Meter to Buy Apartment Outside of Centre']

for column in buy_columns:
    df_buy[column] = pd.to_numeric(df_buy[column], errors='coerce')

df_buy['Average per Square Meter Buy Price'] = df_buy[buy_columns].mean(axis=1)

print(df_buy)

missing_values = df_buy.isnull().sum()
print(missing_values)
#there are missing values here, so I will use the same code from above to fill usin
```

```
                    if found:
                        # Backward filling the missing values using the inflation rates
                        for k in range(j-1, i-1, -1):
                            curr_year = int(df_buy['Year'].iloc[k])
                            if curr_year < 2022:
                                next_year = int(df_buy['Year'].iloc[k + 1])
                                inflation_rate = inflation_dict.get(next_year, 0) / 100
                                next_value /= (1 + inflation_rate)
                            df_buy.iat[k, df_buy.columns.get_loc(column)] = next_value
                    else:  # If no future values are found, like for year 2022, use the
                        for k in range(i - 1, -1, -1):
                            if not pd.isna(df_buy[column].iloc[k]):
                                last_known_value = df_buy[column].iloc[k]
                                df_buy.iat[i, df_buy.columns.get_loc(column)] = last_kr
                                break
```

As you can see above, I had to slightly alter the code in order to account for the last year of 2022 having missing data. In this case, I had to make it so that if it couldn't fill using the next year, it would then fill using the previous year. Luckily, this worked a treat.

In [50]:
```
df_buy
```

Out[50]:

	Year	Price per Square Meter to Buy Apartment in City Centre	Price per Square Meter to Buy Apartment Outside of Centre	Average per Square Meter Buy Price
0	2010	1883.680000	1372.400000	1628.040000
1	2011	2152.780000	1345.490000	1749.135000
2	2012	2152.780000	1119.450000	1636.115000
3	2013	3301.919291	1780.729113	3301.919291
4	2014	3354.750000	1809.220779	3354.750000
5	2015	3408.040000	1811.030000	2609.535000
6	2016	3267.140000	1795.110000	2531.125000
7	2017	3257.040000	1892.120000	2574.580000
8	2018	3762.850000	2116.870000	2939.860000
9	2019	4351.410000	2031.030000	3191.220000
10	2020	6186.710000	2289.420000	4238.065000
11	2021	3632.820000	2389.590000	3011.205000
12	2022	3632.820000	2389.590000	3011.205000

In [51]:
```
df_buy.to_csv('df_buy.csv', index=False)
```

In [52]:
```
df_allhousing=pd.merge(df_rent, df_buy, on='Year', how='outer')
print(df_allhousing)
```

```
    Year  Apartment (1 bedroom) in City Centre  \
0   2010                                924.00
1   2011                               1416.67
2   2012                               1464.22
3   2013                               1433.93
4   2014                               1689.55
5   2015                               1553.70
6   2016                               1753.32
7   2017                               1782.28
8   2018                               1718.58
9   2019                               1934.04
10  2020                               1875.84
11  2021                               1782.90
12  2022                               2045.81

    Apartment (1 bedroom) Outside of Centre  \
0                                    822.00
1                                    881.25
2                                    910.00
3                                    973.18
4                                   1010.45
5                                   1021.95
6                                   1094.77
7                                   1124.15
8                                   1090.68
9                                   1256.51
10                                  1280.85
11                                  1217.50
12                                  1456.40

    Apartment (3 bedrooms) in City Centre  \
0                                 1945.00
1                                 2650.00
2                                 2920.00
3                                 2900.00
4                                 2800.00
5                                 3106.71
6                                 3273.67
7                                 3187.14
8                                 3129.77
9                                 3687.55
10                                3587.95
11                                3319.23
12                                4152.94

    Apartment (3 bedrooms) Outside of Centre  Average Rent Price  \
0                                    1512.50           1300.8750
1                                    1350.00           1574.4800
2                                    1606.25           1725.1175
3                                    2150.00           1864.2775
4                                    1778.17           1819.5425
5                                    1841.43           1880.9475
6                                    1904.22           2006.4950
7                                    2008.43           2025.5000
8                                    1915.20           1963.5575
9                                    2222.75           2275.2125
10                                   2216.53           2240.2925
11                                   2006.67           2081.5750
12                                   2600.00           2563.7875

    Price per Square Meter to Buy Apartment in City Centre  \
0                                         1883.680000
1                                         2152.780000
2                                         2152.780000
3                                         3301.919291
4                                         3354.750000
5                                         3408.040000
6                                         3267.140000
7                                         3257.040000
8                                         3762.850000
9                                         4351.410000
10                                        6186.710000
11                                        3632.820000
12                                        3632.820000

    Price per Square Meter to Buy Apartment Outside of Centre  \
0                                         1372.400000
1                                         1345.490000
2                                         1119.450000
3                                         1780.729113
4                                         1809.220779
5                                         1811.030000
6                                         1795.110000
7                                         1892.120000
8                                         2116.870000
9                                         2031.030000
10                                        2289.420000
11                                        2389.590000
12                                        2389.590000

    Average per Square Meter Buy Price
0                          1628.040000
1                          1749.135000
2                          1636.115000
3                          3301.919291
4                          3354.750000
5                          2609.535000
6                          2531.125000
7                          2574.580000
8                          2939.860000
9                          3191.220000
10                         4238.065000
11                         3011.205000
12                         3011.205000
```

In [53]:
```
df_allhousing.to_csv('df_allhousing.csv', index=False)
```

In [54]:
```
finance_table= soup.find('table', id='tier_8')
headers_finance = [header.text for header in finance_table.find_all('th')]

rows_finance = finance_table.find_all('tr')
table_data = []
for row in rows_finance:
    row_finance_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_finance_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_finance = pd.DataFrame(table_data, columns=headers_finance)

print(df_finance.columns)
```

```
Index(['Year', 'Average Monthly Net Salary (After Tax)'], dtype='object')
```

In [55]:
```
interest_table= soup.find('table', id='tier_14')
headers_interest = [header.text for header in interest_table.find_all('th')]

rows_interest = interest_table.find_all('tr')
table_data = []
for row in rows_interest:
    row_interest_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_interest_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_interest = pd.DataFrame(table_data, columns=headers_interest)

print(df_interest.columns)
```

```
Index(['Year', 'Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate'], dtype='object')
```

In [56]:
```
df_allfinance = pd.merge(df_finance, df_interest, on='Year', how='outer')
print(df_allfinance)
missing_values = df_allfinance.isnull().sum()
print(missing_values)
```

```
    Year Average Monthly Net Salary (After Tax)  \
0   2010                                2750.00
1   2011                                2800.00
2   2012                                3102.86
3   2013                                3686.99
4   2014                                3246.85
5   2015                                3003.10
6   2016                                3578.85
7   2017                                3327.80
8   2018                                4055.27
9   2019                                4211.87
10  2020                                4541.45
11  2021                                5583.44
12  2022                                4976.19

   Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate
0                                                5.63
1                                                5.26
2                                                4.65
3                                                4.37
4                                                4.40
5                                                4.18
6                                                3.96
7                                                3.95
8                                                4.27
9                                                4.27
10                                               4.04
11                                               2.96
12                                               5.63
Year                                                                                0
Average Monthly Net Salary (After Tax)                                              0
Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate          0
dtype: int64
```

In [57]:
```
df_allfinance.to_csv('df_allfinance.csv', index=False)
```

In [58]:
```
transport1_table= soup.find('table', id='tier_9')
headers_transport1 = [header.text for header in transport1_table.find_all('th')]

rows_transport1 = transport1_table.find_all('tr')
table_data = []
for row in rows_transport1:
    row_transport1_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_transport1_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_transport1 = pd.DataFrame(table_data, columns=headers_transport1)

print(df_transport1.columns)
```

```
Index(['Year', 'One-way Ticket (Local Transport)', 'Gasoline (1 liter)'], dtype='object')
```

In [59]:
```
transport2_table= soup.find('table', id='tier_10')
headers_transport2 = [header.text for header in transport2_table.find_all('th')]

rows_transport2 = transport2_table.find_all('tr')
table_data = []
for row in rows_transport2:
    row_transport2_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_transport2_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_transport2 = pd.DataFrame(table_data, columns=headers_transport2)

print(df_transport2.columns)
```

```
Index(['Year', 'Monthly Pass (Regular Price)'], dtype='object')
```

In [60]:
```
df_alltransport=pd.merge(df_transport1, df_transport2, on='Year', how='outer')
print(df_alltransport.columns)

missing_values = df_alltransport.isnull().sum()
print(missing_values)
```

```
Index(['Year', 'One-way Ticket (Local Transport)', 'Gasoline (1 liter)',
       'Monthly Pass (Regular Price)'],
      dtype='object')
Year                              0
One-way Ticket (Local Transport)  0
Gasoline (1 liter)                0
Monthly Pass (Regular Price)      0
dtype: int64
```

In [61]:
```
transport_columns = ['One-way Ticket (Local Transport)', 'Gasoline (1 liter)',
                     'Monthly Pass (Regular Price)']

for column in transport_columns:
    df_alltransport[column] = pd.to_numeric(df_alltransport[column], errors='coerce

df_alltransport['Average Transport Cost'] = df_alltransport[transport_columns].mean


print(df_alltransport)

#For this, it can be assumed that if someone has a monthly public transport pass th
#however, we also do not know how much someone might be driving and spending on gas
#so, I will use the average across all three options for transport to get a relativ
```

```
    Year  One-way Ticket (Local Transport)  Gasoline (1 liter)  \
0   2010                              2.25                0.79
1   2011                              2.25                1.06
2   2012                              2.25                1.12
3   2013                              2.50                1.05
4   2014                              2.25                0.99
5   2015                              2.25                0.77
6   2016                              2.25                0.66
7   2017                              2.25                0.73
8   2018                              2.50                0.84
9   2019                              2.50                0.83
10  2020                              2.50                0.78
11  2021                              2.50                0.90
12  2022                              2.50                1.31

    Monthly Pass (Regular Price)  Average Transport Cost
0                           71.0               24.680000
1                           86.0               29.770000
2                           86.0               29.790000
3                          100.0               34.516667
4                          100.0               34.413333
5                          100.0               34.340000
6                          100.0               34.303333
7                          100.0               34.326667
8                          105.0               36.113333
9                          105.0               36.110000
10                         105.0               36.093333
11                         105.0               36.133333
12                          75.0               26.270000
```

In [62]:
```
df_alltransport.to_csv('df_alltransport.csv', index=False)
```

In [63]:
```
utilities_table= soup.find('table', id='tier_15')
headers_utilities = [header.text for header in utilities_table.find_all('th')]

rows_utilities = utilities_table.find_all('tr')
table_data = []
for row in rows_utilities:
    row_utilities_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_utilities_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_utilities = pd.DataFrame(table_data, columns=headers_utilities)

df_utilities = df_utilities.drop('Mobile Phone Monthly Plan with Calls and 10GB+ Da
#this column was dropped because it was completely blank with no entries on the sit

df_utilities['Total Average Cost of all Utilities'] = df_utilities.drop('Year', axi

print(df_utilities)

df_utilities.to_csv('df_utilities.csv', index=False)
```

```
    Year  \
0   2010
1   2011
2   2012
3   2013
4   2014
5   2015
6   2016
7   2017
8   2018
9   2019
10  2020
11  2021
12  2022

    Basic (Electricity, Heating, Cooling, Water, Garbage) for 85m2 Apartment  \
0                                                      183.33
1                                                      275.00
2                                                      182.08
3                                                      140.98
4                                                      111.78
5                                                      113.05
6                                                      135.72
7                                                      119.05
8                                                      125.25
9                                                      135.28
10                                                     147.24
11                                                     172.86
12                                                     162.68

    Internet (60 Mbps or More, Unlimited Data, Cable/ADSL)  \
0                                                   28.00
1                                                   40.70
2                                                   39.40
3                                                   45.44
4                                                   38.50
5                                                   43.33
6                                                   52.42
7                                                   60.06
8                                                   61.76
9                                                   64.91
10                                                  65.17
11                                                  53.85
12                                                  72.25

    Total Average Cost of all Utilities
0                          183.3328.00
1                          275.0040.70
2                          182.0839.40
3                          140.9845.44
4                          111.7838.50
5                          113.0543.33
6                          135.7252.42
7                          119.0560.06
8                          125.2561.76
9                          135.2864.91
10                         147.2465.17
11                         172.8653.85
12                         162.6872.25
```

In [64]:
```
fun_table= soup.find('table', id='tier_16')
headers_fun = [header.text for header in fun_table.find_all('th')]

rows_fun = fun_table.find_all('tr')
```

```
table_data = []
for row in rows_fun:
    row_fun_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_fun_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_fun = pd.DataFrame(table_data, columns=headers_fun)

print(df_fun)
```

```
    Year Fitness Club, Monthly Fee for 1 Adult  \
0   2010                                34.50
1   2011                                80.00
2   2012                                59.17
3   2013                                39.12
4   2014                                43.28
5   2015                                52.43
6   2016                                57.45
7   2017                                45.26
8   2018                                50.45
9   2019                                49.95
10  2020                                56.78
11  2021                                54.88
12  2022                                64.93

   Tennis Court Rent (1 Hour on Weekend) Cinema, International Release, 1 Seat
0                                      -                                10.00
1                                      -                                12.00
2                                  18.00                                11.50
3                                      -                                10.75
4                                      -                                11.00
5                                  24.40                                12.00
6                                  23.07                                12.00
7                                  22.22                                12.75
8                                  27.22                                12.25
9                                  25.75                                13.00
10                                 17.19                                15.00
11                                 19.00                                12.66
12                                  2.50                                15.00
```

In [65]:
```
for column in df_fun.columns:
    if column != 'Year':
        for i in range(len(df_fun)):
            if df_fun[column].iloc[i] == '-':
                found = False
                for j in range(i + 1, len(df_fun)):
                    if df_fun[column].iloc[j] != '-':
                        found = True
                        next_year = int(df_fun['Year'].iloc[j])
                        next_value = float(df_fun[column].iloc[j])
                        break

                if found:
                    for k in range(j-1, i-1, -1):
                        curr_year = int(df_fun['Year'].iloc[k])
                        next_year = int(df_fun['Year'].iloc[k + 1])

                        inflation_rate = inflation_dict.get(next_year, 0) / 100

                        next_value /= (1 + inflation_rate)

                        df_fun[column].iloc[k] = next_value
```

In [66]:
```
df_fun.columns
```

Out[66]:
```
Index(['Year', 'Fitness Club, Monthly Fee for 1 Adult',
       'Tennis Court Rent (1 Hour on Weekend)',
       'Cinema, International Release, 1 Seat'],
      dtype='object')
```

In [67]:
```
fun_columns = ['Fitness Club, Monthly Fee for 1 Adult',
       'Tennis Court Rent (1 Hour on Weekend)',
       'Cinema, International Release, 1 Seat']
for col in fun_columns:
    df_fun[col] = pd.to_numeric(df_fun[col], errors='coerce')

df_fun['Average Fun Activity Price'] = df_fun[fun_columns].mean(axis=1)

print(df_fun)
#for this, as we do not know what one person might spend on fun, we will take an av
```

```
    Year  Fitness Club, Monthly Fee for 1 Adult  \
0   2010                                34.50
1   2011                                80.00
2   2012                                59.17
3   2013                                39.12
4   2014                                43.28
5   2015                                52.43
6   2016                                57.45
7   2017                                45.26
8   2018                                50.45
9   2019                                49.95
10  2020                                56.78
11  2021                                54.88
12  2022                                64.93

    Tennis Court Rent (1 Hour on Weekend)  \
0                               17.083115
1                               17.629775
2                               18.000000
3                               23.991756
4                               24.375624
5                               24.400000
6                               23.070000
7                               22.220000
8                               27.220000
9                               25.750000
10                              17.190000
11                              19.000000
12                               2.500000

    Cinema, International Release, 1 Seat  Average Fun Activity Price
0                                   10.00                   20.527705
1                                   12.00                   36.543258
2                                   11.50                   29.556667
3                                   10.75                   24.620585
4                                   11.00                   26.218541
5                                   12.00                   29.610000
6                                   12.00                   30.840000
7                                   12.75                   26.743333
8                                   12.25                   29.973333
9                                   13.00                   29.566667
10                                  15.00                   29.656667
11                                  12.66                   28.846667
12                                  15.00                   27.476667
```

In [68]:
```
df_fun.to_csv('df_fun.csv', index=False)
```

In [69]:
```
shopping_table= soup.find('table', id='tier_18')
headers_shopping = [header.text for header in shopping_table.find_all('th')]

rows_shopping = shopping_table.find_all('tr')
table_data = []
for row in rows_shopping:
    row_shopping_data = [td.text.strip() for td in row.find_all('td')]
    table_data.append(row_shopping_data)

table_data = [row for row in table_data if len(row) > 0]
table_data= sorted(table_data, key=lambda x: x[0])
df_shopping = pd.DataFrame(table_data, columns=headers_shopping)

print(df_shopping)
```

```
    Year 1 Pair of Jeans (Levis 501 Or Similar)  \
0   2010                                  25.00
1   2011                                  53.33
2   2012                                  44.00
3   2013                                  43.12
4   2014                                  42.17
5   2015                                  48.06
6   2016                                  48.30
7   2017                                  48.74
8   2018                                  50.32
9   2019                                  52.40
10  2020                                  51.57
11  2021                                  49.06
12  2022                                  61.43

   1 Summer Dress in a Chain Store (Zara, H&M, ...)  \
0                                             16.50
1                                             25.00
2                                             39.17
3                                             31.00
4                                                 -
5                                             37.28
6                                             39.47
7                                             36.88
8                                             37.24
9                                             41.54
10                                            42.44
11                                            34.55
12                                            41.08

   1 Pair of Nike Running Shoes (Mid-Range)  \
0                                      66.67
1                                     100.00
2                                      99.50
3                                      80.00
4                                      81.00
5                                      83.97
6                                      79.07
7                                      78.34
8                                      77.39
9                                      83.56
10                                     84.25
11                                     81.20
12                                     91.80

   1 Pair of Men Leather Business Shoes
0                                150.00
1                                 57.50
2                                103.50
3                                 97.50
4                                101.33
5                                 96.34
6                                104.05
7                                 97.12
8                                110.65
9                                112.45
10                               116.57
11                                94.00
12                               117.22
```

In [70]:
```
inflation_dict = df_inflation1022.set_index('Year')['Ave'].to_dict()

for column in df_shopping.columns:
    if column != 'Year':
        for i in range(len(df_shopping)):
            if df_shopping[column].iloc[i] == '-':
                found = False
                for j in range(i + 1, len(df_shopping)):
                    if df_shopping[column].iloc[j] != '-':
                        found = True
                        next_year = int(df_shopping['Year'].iloc[j])
                        next_value = float(df_shopping[column].iloc[j])
                        break

                if found:
                    for k in range(j-1, i-1, -1):
                        curr_year = int(df_shopping['Year'].iloc[k])
                        next_year = int(df_shopping['Year'].iloc[k + 1])

                        inflation_rate = inflation_dict.get(next_year, 0) / 100

                        next_value /= (1 + inflation_rate)

                        df_shopping[column].iloc[k] = next_value

for column in df_shopping.columns:
    if column != 'Year':
        df_shopping[column] = pd.to_numeric(df_shopping[column], errors='coerce')
```

In [71]:
```
#for this, i'll calcuate the average cost of a clothing item and add in that averag
clothing_columns = [col for col in df_shopping.columns if col != 'Year']

df_shopping['Average Cost of a Clothing Item'] = df_shopping[clothing_columns].mean

print(df_shopping)
```

```
    Year  1 Pair of Jeans (Levis 501 Or Similar)  \
0   2010                                  25.00
1   2011                                  53.33
2   2012                                  44.00
3   2013                                  43.12
4   2014                                  42.17
5   2015                                  48.06
6   2016                                  48.30
7   2017                                  48.74
8   2018                                  50.32
9   2019                                  52.40
10  2020                                  51.57
11  2021                                  49.06
12  2022                                  61.43

    1 Summer Dress in a Chain Store (Zara, H&M, ...)  \
0                                          16.500000
1                                          25.000000
2                                          39.170000
3                                          31.000000
4                                          37.242757
5                                          37.280000
6                                          39.470000
7                                          36.880000
8                                          37.240000
9                                          41.540000
10                                         42.440000
11                                         34.550000
12                                         41.080000

    1 Pair of Nike Running Shoes (Mid-Range)  \
0                                      66.67
1                                     100.00
2                                      99.50
3                                      80.00
4                                      81.00
5                                      83.97
6                                      79.07
7                                      78.34
8                                      77.39
9                                      83.56
10                                     84.25
11                                     81.20
12                                     91.80

    1 Pair of Men Leather Business Shoes  Average Cost of a Clothing Item
0                                 150.00                         64.542500
1                                  57.50                         58.957500
2                                 103.50                         71.542500
3                                  97.50                         62.905000
4                                 101.33                         65.435689
5                                  96.34                         66.412500
6                                 104.05                         67.722500
7                                  97.12                         65.270000
8                                 110.65                         68.900000
9                                 112.45                         72.487500
10                                116.57                         73.707500
11                                 94.00                         64.702500
12                                117.22                         77.882500
```

In [72]:
```
df_shopping.to_csv('df_shopping.csv', index=False)
```

*Now I have the following data frames from web scraping (well, I have quite a few, but I have combined them into the following areas): df_shopping df_fun df_utiltiies df_allhousing df_alltransport df_allfooddrink df_allmarket df_allfinance

In [73]:
```
%who DataFrame
```

```
crime_data_1022  crime_data_large        crimes_1022_small       crimes_year     d
f_allfinance     df_allfooddrink         df_allhousing   df_allmarket    df_alltra
nsport
df_buy   df_combined_produce     df_drink        df_finance      df_fun  df_inflat
ion      df_inflation1022        df_interest     df_market
df_produce       df_produce2     df_rent         df_rest         df_shopping     d
f_transport1     df_transport2   df_utilities    grouped_categories
```

Now, I will use my all dataframes to make a large data frame encompassing all finance, market, food and drink, transport, housing, utilities, fun and shopping data sets.

In [74]:
```
dataframes = [df_allfinance, df_allhousing, df_utilities, df_alltransport, df_allfo

df_cost_of_living = reduce(lambda left, right: pd.merge(left, right, on='Year', how

df_cost_of_living
```

Out[74]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Apartment (1 bedroom) in City Centre	Apartment (1 bedroom) Outside of Centre	Apartment (3 bedrooms) in City Centre	Apartment (3 bedrooms) Outside of Centre	Average Rent Price	Pr M Apa
0	2010	2750.00	5.63	924.00	822.00	1945.00	1512.50	1300.8750	1883.
1	2011	2800.00	5.26	1416.67	881.25	2650.00	1350.00	1574.4800	2152.
2	2012	3102.86	4.65	1464.22	910.00	2920.00	1606.25	1725.1175	2152.
3	2013	3686.99	4.37	1433.93	973.18	2900.00	2150.00	1864.2775	3301.
4	2014	3246.85	4.40	1689.55	1010.45	2800.00	1778.17	1819.5425	3354.
5	2015	3003.10	4.18	1553.70	1021.95	3106.71	1841.43	1880.9475	3408.
6	2016	3578.85	3.96	1753.32	1094.77	3273.67	1904.22	2006.4950	3267.
7	2017	3327.80	3.95	1782.28	1124.15	3187.14	2008.43	2025.5000	3257.
8	2018	4055.27	4.27	1718.58	1090.68	3129.77	1915.20	1963.5575	3762.
9	2019	4211.87	4.27	1934.04	1256.51	3687.55	2222.75	2275.2125	4351.
10	2020	4541.45	4.04	1875.84	1280.85	3587.95	2216.53	2240.2925	6186.
11	2021	5583.44	2.96	1782.90	1217.50	3319.23	2006.67	2081.5750	3632.
12	2022	4976.19	5.63	2045.81	1456.40	4152.94	2600.00	2563.7875	3632.

13 rows × 54 columns

In [75]:
```
print(df_cost_of_living.columns)
```

```
Index(['Year', 'Average Monthly Net Salary (After Tax)',
       'Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rat
e',
       'Apartment (1 bedroom) in City Centre',
       'Apartment (1 bedroom) Outside of Centre',
       'Apartment (3 bedrooms) in City Centre',
       'Apartment (3 bedrooms) Outside of Centre', 'Average Rent Price',
       'Price per Square Meter to Buy Apartment in City Centre',
       'Price per Square Meter to Buy Apartment Outside of Centre',
       'Average per Square Meter Buy Price',
       'Basic (Electricity, Heating, Cooling, Water, Garbage) for 85m2 Apartment',
       'Internet (60 Mbps or More, Unlimited Data, Cable/ADSL)',
       'Total Average Cost of all Utilities',
       'One-way Ticket (Local Transport)', 'Gasoline (1 liter)',
       'Monthly Pass (Regular Price)', 'Average Transport Cost',
       'Meal, Inexpensive Restaurant',
       'Meal for 2 People, Mid-range Restaurant, Three-course',
       'McMeal at McDonalds (or Equivalent Combo Meal)',
       'Average Resturaunt Meal Price', 'Domestic_Beer_0.5_liter_draught',
       'Imported_Beer_0.33_liter_bottle', 'Coke/Pepsi_0.33_liter_bottle',
       'Water_0.33_liter_bottle_', 'Cappuccino_regular', 'Average Drink Price',
       'Apples (1kg)', 'Oranges (1kg)', 'Potato (1kg)', 'Lettuce (1 head)',
       'Rice (white), (1kg)', 'Tomato (1kg)', 'Banana (1kg)', 'Onion (1kg)',
       'Local Cheese (1kg)', 'Bottle of Wine (Mid-Range)',
       'Cigarettes 20 Pack (Marlboro)', 'Chicken Fillets (1kg)',
       'Beef Round (1kg) (or Equivalent Back Leg Red Meat)',
       'Milk (regular), (1 liter)', 'Loaf of Fresh White Bread (500g)',
       'Eggs (regular) (12)', 'Total Average Cost of all Groceries',
       '1 Pair of Jeans (Levis 501 Or Similar)',
       '1 Summer Dress in a Chain Store (Zara, H&M, ...)',
       '1 Pair of Nike Running Shoes (Mid-Range)',
       '1 Pair of Men Leather Business Shoes',
       'Average Cost of a Clothing Item',
       'Fitness Club, Monthly Fee for 1 Adult',
       'Tennis Court Rent (1 Hour on Weekend)',
       'Cinema, International Release, 1 Seat', 'Average Fun Activity Price'],
      dtype='object')
```

In [76]:
```
missing_values_cost = df_cost_of_living.isna().sum()
print(missing_values_cost)
```

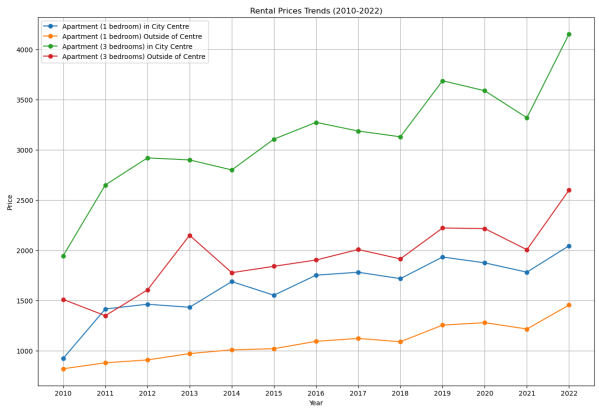
```
Year                                                                    0
Average Monthly Net Salary (After Tax)                                  0
Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate    0
Apartment (1 bedroom) in City Centre                                    0
Apartment (1 bedroom) Outside of Centre                                 0
Apartment (3 bedrooms) in City Centre                                   0
Apartment (3 bedrooms) Outside of Centre                                0
Average Rent Price                                                      0
Price per Square Meter to Buy Apartment in City Centre                  0
Price per Square Meter to Buy Apartment Outside of Centre               0
Average per Square Meter Buy Price                                      0
Basic (Electricity, Heating, Cooling, Water, Garbage) for 85m2 Apartment    0
Internet (60 Mbps or More, Unlimited Data, Cable/ADSL)                  0
Total Average Cost of all Utilities                                     0
One-way Ticket (Local Transport)                                        0
Gasoline (1 liter)                                                      0
Monthly Pass (Regular Price)                                            0
Average Transport Cost                                                  0
Meal, Inexpensive Restaurant                                            0
Meal for 2 People, Mid-range Restaurant, Three-course                   0
McMeal at McDonalds (or Equivalent Combo Meal)                          0
Average Resturaunt Meal Price                                           0
Domestic_Beer_0.5_liter_draught                                         0
Imported_Beer_0.33_liter_bottle                                         0
Coke/Pepsi_0.33_liter_bottle                                            0
Water_0.33_liter_bottle_                                                0
Cappuccino_regular                                                      0
Average Drink Price                                                     0
Apples (1kg)                                                            0
Oranges (1kg)                                                           0
Potato (1kg)                                                            0
Lettuce (1 head)                                                        0
Rice (white), (1kg)                                                     0
Tomato (1kg)                                                            0
Banana (1kg)                                                            0
Onion (1kg)                                                             0
Local Cheese (1kg)                                                      0
Bottle of Wine (Mid-Range)                                              0
Cigarettes 20 Pack (Marlboro)                                           0
Chicken Fillets (1kg)                                                   0
Beef Round (1kg) (or Equivalent Back Leg Red Meat)                      0
Milk (regular), (1 liter)                                               0
Loaf of Fresh White Bread (500g)                                        0
Eggs (regular) (12)                                                     0
Total Average Cost of all Groceries                                     0
1 Pair of Jeans (Levis 501 Or Similar)                                  0
1 Summer Dress in a Chain Store (Zara, H&M, ...)                        0
1 Pair of Nike Running Shoes (Mid-Range)                                0
1 Pair of Men Leather Business Shoes                                    0
Average Cost of a Clothing Item                                         0
Fitness Club, Monthly Fee for 1 Adult                                   0
Tennis Court Rent (1 Hour on Weekend)                                   0
Cinema, International Release, 1 Seat                                   0
Average Fun Activity Price                                              0
dtype: int64
```

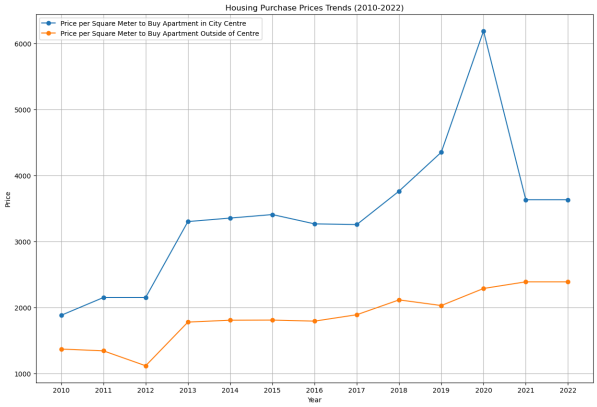
In [77]:
```
df_cost_of_living.to_csv('df_cost_of_living.csv', index=False)
```

1.7.1 Visualisations for Cost of Living Web Scraped Data

1. Here, I will look at the trends for rental prices and purchase prices for housing over time.

In [78]:
```
plt.figure(figsize=(15, 10))

for column in df_allhousing.columns:
    if column in ('Apartment (1 bedroom) in City Centre',
                  'Apartment (1 bedroom) Outside of Centre',
                  'Apartment (3 bedrooms) in City Centre',
                  'Apartment (3 bedrooms) Outside of Centre'):
        plt.plot(df_allhousing['Year'], df_allhousing[column], marker='o', label=co

plt.title('Rental Prices Trends (2010-2022)')
plt.xlabel('Year')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(15, 10))

for column in df_allhousing.columns:
    if column in ('Price per Square Meter to Buy Apartment in City Centre',
       'Price per Square Meter to Buy Apartment Outside of Centre'):
        plt.plot(df_allhousing['Year'], df_allhousing[column], marker='o', label=co

plt.title('Housing Purchase Prices Trends (2010-2022)')
plt.xlabel('Year')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()
```





1.7.2 Combining Cost of Living and Crime Data Sets

Now that I have all of this data for cost of living and crime, I will combine them into one large data set for future analysis. This will be quite large, so I will also make a smaller one that includes less crime data as the important information to answer my research question is the crime type or category and the year. So, my smaller data set will include this information and the averages for the cost of living categories in order to make analysis easier in the future.

In [79]:
```
df_cost_of_living['Year'] = df_cost_of_living['Year'].astype(int)
crimes_1022_small['Year'] = crimes_1022_small['Year'].astype(int)

crimes_and_costofliving_df = pd.merge(df_cost_of_living, crimes_1022_small, on='Yea
print(crimes_and_costofliving_df.head())
```

```
   Year  Average Monthly Net Salary (After Tax)  \
0  2010                                 2750.00
1  2010                                 2750.00
2  2010                                 2750.00
3  2010                                 2750.00
4  2010                                 2750.00

   Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate  \
0                                               5.63
1                                               5.63
2                                               5.63
3                                               5.63
4                                               5.63

   Apartment (1 bedroom) in City Centre  \
0                                 924.0
1                                 924.0
2                                 924.0
3                                 924.0
4                                 924.0

   Apartment (1 bedroom) Outside of Centre  \
0                                    822.0
1                                    822.0
2                                    822.0
3                                    822.0
4                                    822.0

   Apartment (3 bedrooms) in City Centre  \
0                                 1945.0
1                                 1945.0
2                                 1945.0
3                                 1945.0
4                                 1945.0

   Apartment (3 bedrooms) Outside of Centre  Average Rent Price  \
0                                    1512.5            1300.875
1                                    1512.5            1300.875
2                                    1512.5            1300.875
3                                    1512.5            1300.875
4                                    1512.5            1300.875

   Price per Square Meter to Buy Apartment in City Centre  \
0                                            1883.68
1                                            1883.68
2                                            1883.68
3                                            1883.68
4                                            1883.68

   Price per Square Meter to Buy Apartment Outside of Centre  ...         ID  \
0                                             1372.4          ...  11649978
1                                             1372.4          ...  11649965
2                                             1372.4          ...  11609751
3                                             1372.4          ...  11610666
4                                             1372.4          ...   7823660

  Case Number                Date                Block        Primary Type  \
0    JC217323 2010-04-27 09:45:00      009XX W 72ND ST  DECEPTIVE PRACTICE
1    JC217381 2010-11-09 03:30:00  045XX S CALUMET AVE  DECEPTIVE PRACTICE
2    JC169189 2010-07-30 12:00:00      043XX S WOOD ST  DECEPTIVE PRACTICE
3    JC169916 2010-06-01 02:00:00   049XX W MADISON ST  DECEPTIVE PRACTICE
4    HS634379 2010-11-19 09:00:00  076XX S ABERDEEN ST             ROBBERY

                        Description  Arrest  Domestic  District  \
0  FINANCIAL IDENTITY THEFT OVER $ 300   False     False       7.0
1  FINANCIAL IDENTITY THEFT OVER $ 300   False     False       2.0
2  FINANCIAL IDENTITY THEFT OVER $ 300   False     False       9.0
3  FINANCIAL IDENTITY THEFT OVER $ 300   False     False      15.0
4              STRONG ARM - NO WEAPON    True     False       6.0

                 Category
0  Economically Motivated
1  Economically Motivated
2  Economically Motivated
3  Economically Motivated
4  Economically Motivated

[5 rows x 64 columns]
```

In [80]:
```
print(crimes_and_costofliving_df.columns)
```

```
Index(['Year', 'Average Monthly Net Salary (After Tax)',
       'Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rat
e',
       'Apartment (1 bedroom) in City Centre',
       'Apartment (1 bedroom) Outside of Centre',
       'Apartment (3 bedrooms) in City Centre',
       'Apartment (3 bedrooms) Outside of Centre', 'Average Rent Price',
       'Price per Square Meter to Buy Apartment in City Centre',
       'Price per Square Meter to Buy Apartment Outside of Centre',
       'Average per Square Meter Buy Price',
       'Basic (Electricity, Heating, Cooling, Water, Garbage) for 85m2 Apartment',
       'Internet (60 Mbps or More, Unlimited Data, Cable/ADSL)',
       'Total Average Cost of all Utilities',
       'One-way Ticket (Local Transport)', 'Gasoline (1 liter)',
       'Monthly Pass (Regular Price)', 'Average Transport Cost',
       'Meal, Inexpensive Restaurant',
       'Meal for 2 People, Mid-range Restaurant, Three-course',
       'McMeal at McDonalds (or Equivalent Combo Meal)',
       'Average Resturaunt Meal Price', 'Domestic_Beer_0.5_liter_draught',
       'Imported_Beer_0.33_liter_bottle', 'Coke/Pepsi_0.33_liter_bottle',
       'Water_0.33_liter_bottle_', 'Cappuccino_regular', 'Average Drink Price',
       'Apples (1kg)', 'Oranges (1kg)', 'Potato (1kg)', 'Lettuce (1 head)',
       'Rice (white), (1kg)', 'Tomato (1kg)', 'Banana (1kg)', 'Onion (1kg)',
       'Local Cheese (1kg)', 'Bottle of Wine (Mid-Range)',
       'Cigarettes 20 Pack (Marlboro)', 'Chicken Fillets (1kg)',
       'Beef Round (1kg) (or Equivalent Back Leg Red Meat)',
       'Milk (regular), (1 liter)', 'Loaf of Fresh White Bread (500g)',
       'Eggs (regular) (12)', 'Total Average Cost of all Groceries',
       '1 Pair of Jeans (Levis 501 Or Similar)',
       '1 Summer Dress in a Chain Store (Zara, H&M, ...)',
       '1 Pair of Nike Running Shoes (Mid-Range)',
       '1 Pair of Men Leather Business Shoes',
       'Average Cost of a Clothing Item',
       'Fitness Club, Monthly Fee for 1 Adult',
       'Tennis Court Rent (1 Hour on Weekend)',
       'Cinema, International Release, 1 Seat', 'Average Fun Activity Price',
       'ID', 'Case Number', 'Date', 'Block', 'Primary Type', 'Description',
       'Arrest', 'Domestic', 'District', 'Category'],
      dtype='object')
```

In [85]:
```
crimes_and_costofliving_df.to_csv("crimes_and_costofliving.csv", index=False)
```

In [82]:
```
columns_to_drop = [
    'Apartment (1 bedroom) in City Centre',
    'Apartment (1 bedroom) Outside of Centre',
    'Apartment (3 bedrooms) in City Centre',
    'Apartment (3 bedrooms) Outside of Centre',
    'Price per Square Meter to Buy Apartment in City Centre',
    'Price per Square Meter to Buy Apartment Outside of Centre',
    'Basic (Electricity, Heating, Cooling, Water, Garbage) for 85m2 Apartment',
    'Internet (60 Mbps or More, Unlimited Data, Cable/ADSL)',
    'One-way Ticket (Local Transport)', 'Gasoline (1 liter)',
    'Monthly Pass (Regular Price)', 'Average Transport Cost',
    'Meal, Inexpensive Restaurant',
    'Meal for 2 People, Mid-range Restaurant, Three-course',
    'McMeal at McDonalds (or Equivalent Combo Meal)',
    'Domestic_Beer_0.5_liter_draught',
    'Imported_Beer_0.33_liter_bottle', 'Coke/Pepsi_0.33_liter_bottle',
    'Water_0.33_liter_bottle_', 'Cappuccino_regular',
    'Apples (1kg)', 'Oranges (1kg)', 'Potato (1kg)', 'Lettuce (1 head)',
    'Rice (white), (1kg)', 'Tomato (1kg)', 'Banana (1kg)', 'Onion (1kg)',
    'Local Cheese (1kg)', 'Bottle of Wine (Mid-Range)',
    'Cigarettes 20 Pack (Marlboro)', 'Chicken Fillets (1kg)',
    'Beef Round (1kg) (or Equivalent Back Leg Red Meat)',
    'Milk (regular), (1 liter)', 'Loaf of Fresh White Bread (500g)',
    'Eggs (regular) (12)',
    '1 Pair of Jeans (Levis 501 Or Similar)',
    '1 Summer Dress in a Chain Store (Zara, H&M, ...)',
    '1 Pair of Nike Running Shoes (Mid-Range)',
    '1 Pair of Men Leather Business Shoes',
    'Fitness Club, Monthly Fee for 1 Adult',
    'Tennis Court Rent (1 Hour on Weekend)',
    'Cinema, International Release, 1 Seat',
    'ID','Case Number', 'Date', 'Block', 'Domestic', 'District'
]

df_small_crimes_and_costofliving = crimes_and_costofliving_df.drop(columns=columns_

print(df_small_crimes_and_costofliving.head())

df_small_crimes_and_costofliving.columns
```

```
   Year Average Monthly Net Salary (After Tax)  \
0  2010                                2750.00
1  2010                                2750.00
2  2010                                2750.00
3  2010                                2750.00
4  2010                                2750.00

  Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate  \
0                                              5.63
1                                              5.63
2                                              5.63
3                                              5.63
4                                              5.63

   Average Rent Price  Average per Square Meter Buy Price  \
0           1300.875                            1628.04
1           1300.875                            1628.04
2           1300.875                            1628.04
3           1300.875                            1628.04
4           1300.875                            1628.04

  Total Average Cost of all Utilities  Average Resturaunt Meal Price  \
0                        183.3328.00                       19.333333
1                        183.3328.00                       19.333333
2                        183.3328.00                       19.333333
3                        183.3328.00                       19.333333
4                        183.3328.00                       19.333333

   Average Drink Price  Total Average Cost of all Groceries  \
0             3.2025                            79.982082
1             3.2025                            79.982082
2             3.2025                            79.982082
3             3.2025                            79.982082
4             3.2025                            79.982082

   Average Cost of a Clothing Item  Average Fun Activity Price  \
0                          64.5425                   20.527705
1                          64.5425                   20.527705
2                          64.5425                   20.527705
3                          64.5425                   20.527705
4                          64.5425                   20.527705

         Primary Type                          Description  Arrest  \
0  DECEPTIVE PRACTICE  FINANCIAL IDENTITY THEFT OVER $ 300   False
1  DECEPTIVE PRACTICE  FINANCIAL IDENTITY THEFT OVER $ 300   False
2  DECEPTIVE PRACTICE  FINANCIAL IDENTITY THEFT OVER $ 300   False
3  DECEPTIVE PRACTICE  FINANCIAL IDENTITY THEFT OVER $ 300   False
4            ROBBERY                STRONG ARM - NO WEAPON    True

               Category
0  Economically Motivated
1  Economically Motivated
2  Economically Motivated
3  Economically Motivated
4  Economically Motivated
```

Out[82]:
```
Index(['Year', 'Average Monthly Net Salary (After Tax)',
       'Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate',
       'Average Rent Price', 'Average per Square Meter Buy Price',
       'Total Average Cost of all Utilities', 'Average Resturaunt Meal Price',
       'Average Drink Price', 'Total Average Cost of all Groceries',
       'Average Cost of a Clothing Item', 'Average Fun Activity Price',
       'Primary Type', 'Description', 'Arrest', 'Category'],
      dtype='object')
```

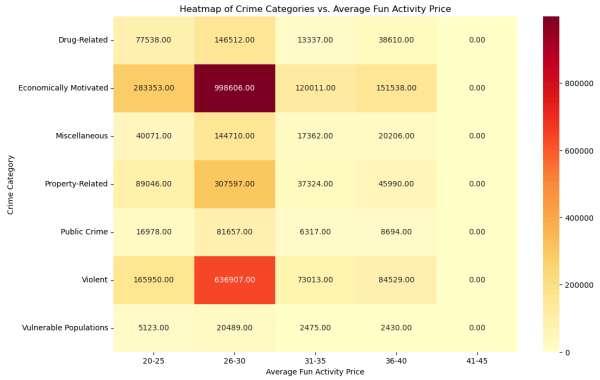
In [83]:
```
df_small_crimes_and_costofliving.to_csv("small_crimes_and_costofliving.csv", index=
```

1.7.3 Visualisations for the Combined Cost of Living and Crimes Data Set

Here, I am creating heatmaps that help to identify correlations between various prices, like the average rent price or fun activity price, and the number of crimes committed in each category.

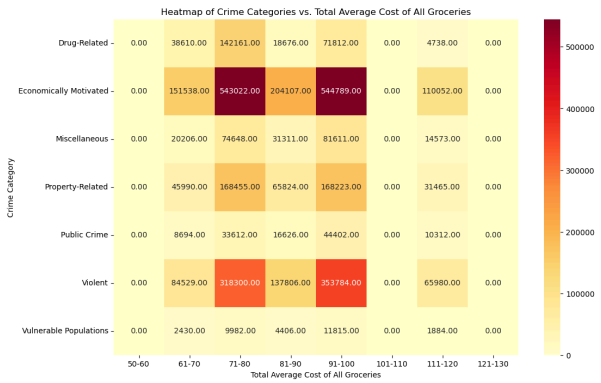
From these, we can already start to observe some strong correlation which should make for an interesting further analysis.

In [84]:
```
bins = [20,25,30,35,40,45]
labels = ['20-25', '26-30', '31-35', '36-40', '41-45']
df_small_crimes_and_costofliving['Fun Cost Bin'] = pd.cut(df_small_crimes_and_costo

pivot_table = df_small_crimes_and_costofliving.pivot_table(index='Category', columr

plt.figure(figsize=(12, 8))
sb.heatmap(pivot_table, annot=True, cmap='YlOrRd', fmt='.2f')

plt.title('Heatmap of Crime Categories vs. Average Fun Activity Price')
plt.xlabel('Average Fun Activity Price')
plt.ylabel('Crime Category')
plt.show()
```

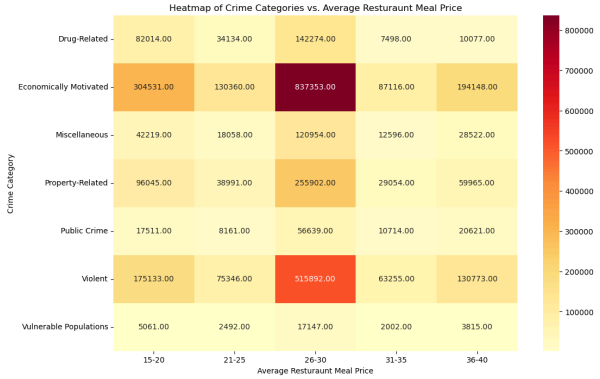


In [86]:
```
bins = [50, 60, 70, 80, 90, 100, 110, 120, 130]
labels = ['50-60', '61-70', '71-80', '81-90', '91-100', '101-110', '111-120', '121-
df_small_crimes_and_costofliving['Grocery Cost Bin'] = pd.cut(df_small_crimes_and_c

pivot_table = df_small_crimes_and_costofliving.pivot_table(index='Category', columr

plt.figure(figsize=(12, 8))
sb.heatmap(pivot_table, annot=True, cmap='YlOrRd', fmt='.2f')
```

```
plt.title('Heatmap of Crime Categories vs. Total Average Cost of All Groceries')
plt.xlabel('Total Average Cost of All Groceries')
plt.ylabel('Crime Category')
plt.show()
```



In [87]:
```
bins = [15, 20, 25, 30, 35, 40]
labels = ['15-20', '21-25', '26-30', '31-35', '36-40']
df_small_crimes_and_costofliving['Resturaunt Cost Bin'] = pd.cut(df_small_crimes_ar

pivot_table = df_small_crimes_and_costofliving.pivot_table(index='Category', columr

plt.figure(figsize=(12, 8))
sb.heatmap(pivot_table, annot=True, cmap='YlOrRd', fmt='.2f')

plt.title('Heatmap of Crime Categories vs. Average Resturaunt Meal Price')
plt.xlabel('Average Resturaunt Meal Price')
plt.ylabel('Crime Category')
plt.show()
```



1.8 Summary of Prepared Data

As we can see from the visualisations, crime levels have decreased over time while cost of living has increased over that same time period. Notably, the category of crime 'Economically Motivated Crimes', is the highest over the years. Further analysis is needed to statistically see if the rise of cost of living impacts the crime levels at all, if certain categories of cost of living increases impact certain categories of crime at all, or any other relevant correllations. From the derived data sets, crimes_and_costofliving, this statistical analysis will be possible. Rising economic pressures are visible, however, from this exploratory analysis, a rise in crime does not seem to coincide. Perhaps society is too busy trying to afford the cost of living to 'eat the rich'.

1.9 References and Resources

1.9.1 References

1. Becker, G.S. (1968) 'Crime and punishment: An economic approach', The Economic Dimensions of Crime, pp. 13–68. doi:10.1007/978-1-349-62853-7_2.
2. Fuscaldo, D. (2022) How to adjust fixed income spending for inflation, AARP. Available at: https://www.aarp.org/retirement/planning-for-retirement/info-2022/inflation-adjusted-living-expenses.html (Accessed: 02 November 2023).
3. Kang, S. (2016) 'Inequality and crime revisited: Effects of local inequality and economic segregation on crime', Journal of Population Economics, 29(2), pp. 593–626. doi:10.1007/s00148-015-0579-3.

4. Nicholson, K. (2022) Has the cost of living crisis had an impact on crime levels?, HuffPost UK. Available at: https://www.huffingtonpost.co.uk/entry/cost-of-living-crisis-impact-on-crime_uk_636a2b7be4b05f221e7da88d (Accessed: 02 November 2023).

1.9.2 Resources

- Web Scraping:
  - Data Programming Lectures, Labs, and Videos: Sean McGrath, Goldsmiths University of London
  - Web scraping a table using Beautifulsoup tutorial, https://www.youtube.com/watch?v=T1qv3ksMDq4
  - One on One tutoring sessions: Brian DeWeese, Senior Software Engineer at Cboe Global Markets
- Data Cleaning and Processing:
  - Lectures, Labs, and Videos: Sean McGrath, Goldsmiths University of London
  - Data cleaning tutorial using Python, https://www.youtube.com/watch?v=qxpKCBV60U4
  - Data cleaning tutorial using Python, https://www.youtube.com/watch?v=4yl3vVe0Jos
- Exploratory Analysis:
  - Data Visualisation Lectures, Labs, and Videos: Joy Eze, Goldsmiths University of London
  - One on One tutoring sessions: Brian DeWeese, Senior Software Engineer at Cboe Global Markets